Journal Title IJRR
XX(X):1–19
©The Author(s) 2016
Reprints and permission:
sagepub.co.uk/journalsPermissions.nav
DOI: 10.1177/ToBeAssigned
www.sagepub.com/
SAGE

Convex Risk Bounded Continuous-Time Trajectory Planning and Tube Design in Uncertain Nonconvex Environments

Ashkan Jasour*, Weiqiao Han*, and Brian Williams

Abstract

In this paper, we address the trajectory planning problem in uncertain nonconvex static and dynamic environments that contain obstacles with probabilistic location, size, and geometry. To address this problem, we provide a risk bounded trajectory planning method that looks for continuous-time trajectories with guaranteed bounded risk over the planning time horizon. Risk is defined as the probability of collision with uncertain obstacles. Existing approaches to address risk bounded trajectory planning problems either are limited to Gaussian uncertainties and convex obstacles or rely on sampling-based methods that need uncertainty samples and time discretization. To address the risk bounded trajectory planning problem, we leverage the notion of risk contours to transform the risk bounded planning problem into a deterministic optimization problem. Risk contours are the set of all points in the uncertain environment with guaranteed bounded risk. The obtained deterministic optimization is, in general, nonlinear and nonconvex time-varying optimization. We provide convex methods based on sum-of-squares optimization to efficiently solve the obtained nonconvex time-varying optimization problem and obtain the continuous-time risk bounded trajectories without time discretization. The provided approach deals with arbitrary (and known) probabilistic uncertainties, nonconvex and nonlinear, static and dynamic obstacles, and is suitable for online trajectory planning problems. In addition, we provide convex methods based on sum-of-squares optimization to build the max-sized tube with respect to its parameterization along the trajectory so that any state inside the tube is guaranteed to have bounded risk.

Keywords

Motion planning, automation, sum-of-squares optimization

arXiv:2305.17291v2 [cs.AI] 4 Jun 2023

1 Introduction

In order for robots to navigate safely in the real world, they need to plan safe trajectories to avoid static and moving obstacles, such as humans and vehicles, under perception uncertainties. The motion planning problem in dynamic environments is known to be computationally hard Reif and Sharir (1994). In this paper, we address the trajectory planning problem in uncertain nonconvex static and dynamic environments that contain obstacles with probabilistic location, size, and geometry. In this problem, the time-varying, nonconvex, and probabilistic nature of the obstacle-free safe regions makes the trajectory planning problem challenging.

Several approaches have been proposed to address the trajectory planning problems. In the absence of obstacles, one can use standard convex optimization to look for polynomial trajectories that satisfy boundary and way-points conditions Lynch and Park (2017); Kumar et al. (1999); Charles et al. (2016.). In the presence of obstacles, sampling-based methods, including rapidly exploring random tree (RRT) and probabilistic roadmap (PRM), and virtual potential field methods are widely used to find obstacle-free trajectories LaValle (2006); Lynch and Park (2017). In Deits and Tedrake (2015), a mixed-integer optimization is provided for trajectory planning in the presence of convex obstacles. The proposed method first uses convex segmentation to compute convex regions of obstacle-free space. Then, it uses a mixed-integer optimization to assign polynomial trajectories to the computed convex safe regions. Also, Khadir et al. (2021) provides a moment-sum-of-squares-based convex optimization to obtain piece-wise linear trajectories in the presence of deterministic time-varying polynomial obstacles without the need for time discretization.

Trajectory planning problems under uncertainty look for trajectories with a bounded probability of collision with uncertain obstacles. Existing methods to address trajectory planning problems under uncertainty either are limited to Gaussian uncertainties and convex obstacles Blackmore and Ono (2009); Blackmore et al. (2010); Schwarting et al. (2017); Luders et al. (2010); Axelrod et al. (2018); Dawson et al. (2020); Dai et al. (2019) or rely on sampling-based methods Cannon (2017); Calafiore and Campi (2006); Janson et al. (2018). For example, chance constrained RRT* algorithm in Luders et al. (2010) assumes Gaussian uncertainties and linear obstacles and performs a probabilistic collision check for the nodes of the search tree. Hence, it can not guarantee to satisfy the probabilistic safety constraints along the edges of the search tree. The Monte Carlo-based motion planning algorithms, e.g., Janson et al.

MIT, Computer Science and Artificial Intelligence Laboratory.
* These authors contributed equally to the paper.

Corresponding author:
Weiqiao Han, 32 Vassar Street, Rm 235, Cambridge, MA 02139.
Email: weiqiaoh@mit.edu

(2018), use a large number of uncertainty samples to estimate the probability of collision of a given trajectory. Sampling-based methods do not provide any analytical bounds on the probability of collision. Instead providing such a certificate usually requires the number of samples to go to infinity. Due to a large number of samples, sampling-based methods sometimes can be computationally intractable.

Also, Summers (2018); Wang et al. (2020) use moment-based approaches to address non-Gaussian uncertainties in motion planning problems in the presence of convex obstacles. More precisely, to obtain the risk bounded trajectories, Summers (2018) uses first and second-order moments of uncertainties and RRT* algorithm in the presence of linear obstacles and Wang et al. (2020) uses higher-order moments and interior-point nonlinear optimization solvers in the presence of ellipsoidal obstacles.

Statement of Contributions: In this paper, we propose novel convex algorithms for risk bounded continuous-time trajectory planning in uncertain nonconvex static and dynamic environments that contain obstacles with probabilistic location, size, geometry, and trajectories with arbitrary probabilistic distributions. To achieve risk bounded plans:

(1) We provide an analytical method to compute risk contours maps. Risk contours allow us to identify risk bounded regions in uncertain environments and transform nonlinear stochastic planning problems into deterministic standard planning problems, in the presence of arbitrary (and known) probabilistic uncertainties. Hence, standard deterministic motion planning algorithms, e.g., RRT*, PRM, can be employed to look for safe (risk-bounded) plans.

(2) To solve the obtained deterministic planning problems, we provide two planners including i) sum-of-squares-based RRT algorithm and ii) sum-of-squares-based convex optimization that allows us to theoretically look for global optimal plans in nonconvex environments.

(3) We also provide continuous-time safety guarantees in stochastic environments. To ensure safety, existing planning under uncertainty algorithms "only" verify the safety of a finite set of waypoints (time-discretization). Unlike the existing planners, the provided planners of this paper ensure the (risk bounded) safety of the continuous-time trajectories without the need for time discretization.

(4) In addition to planning trajectories, we build tubes around the trajectory so that any states inside the tube are guaranteed to have bounded risk. This is achieved via the following two sub-procedures:

- Verifying the safety of the tube via convex programming.
- Looking for the max-sized tube with respect to any given parameterization of the tube using binary search.

(5) We compare our methods with other methods, such as the linear Gaussian method and the Chance-Constrained RRT method. Specifically, we compare both our analytical method for computing risk contours and our trajectory planning method with the linear Gaussian method. And we compare our our tube design method with the Chance-Constrained RRT method.

This article is in continuation of our previous work presented in Jasour et al. (2021a) and the contributions (4) and (5) specifically are the extensions. The outline of the paper is as follows: Section 2 presents the notation adopted in the paper and definitions of polynomials, moments, and sum-of-squares optimization. In Section 3, we provide the problem formulation of risk bounded continuous-time trajectory planning. Section 4 provides analytical approaches to compute static and dynamic risk contours defined for static and dynamic uncertain obstacles to identify the risk bounded safe regions in uncertain environments. In Section 5, using the obtained risk contours, we provide sum-of-squares-based planners to look for continuous-time risk bounded trajectories in uncertain static and dynamic environments. In Section 6, we extend planning continuous-time risk bounded trajectories to planning continuous-time risk bounded tubes, and we search for the max-sized tube with respect to the given parameterization based on sum-of-squares programming and the binary search algorithm. In Section 7, we present experimental results on the risk bounded planning problems of autonomous and robotic systems followed by a discussion section. Finally, concluding remarks and future work are given in Section 8.

2 Notations and definitions

This section covers notation and some basic definitions of polynomials, moments of probability distributions, and sum-of-squares optimization. For a vector $\mathbf{x} \in \mathbb{R}^n$ and multi-index $\boldsymbol{\alpha} \in \mathbb{N}^n$, let $x^{\boldsymbol{\alpha}} = \prod_{i=1}^n x_i^{\alpha_i}$.

Polynomials: Let $\mathbb{R}[x]$ be the set of real polynomials in the variables $\mathbf{x} \in \mathbb{R}^n$. Given polynomial $\mathcal{P}(\mathbf{x}) : \mathbb{R}^n \to \mathbb{R}$, we represent $\mathcal{P}$ as $\sum_{\boldsymbol{\alpha} \in \mathbb{N}^n} p_{\boldsymbol{\alpha}} x^{\boldsymbol{\alpha}}$ where $\{x^{\boldsymbol{\alpha}}\}_{\boldsymbol{\alpha} \in \mathbb{N}^n}$ are the standard monomial basis of $\mathbb{R}[x]$, $\mathbf{p} = \{p_{\boldsymbol{\alpha}}\}_{\boldsymbol{\alpha} \in \mathbb{N}^n}$ denotes the coefficients, and $\boldsymbol{\alpha} \in \mathbb{N}^n$. In this paper, we use polynomials to describe uncertain obstacles and continuous-time trajectories. For example the set $\{(x_1, x_2) : 1 - (x_1 - \omega_1)^2 - (x_2 - \omega_2)^2 \geq 0\}$ represents a circle-shaped obstacle whose center is subjected to uncertainty modeled with random variables (ω_1, ω_2). Also, $\begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} t + \begin{bmatrix} 0.5 \\ 2 \end{bmatrix} t^2, t \in [0, 1]$ is an example of polynomial trajectory of order 2 in 2D environment between the points $\mathbf{x}(0) = (0, 0)$ and $\mathbf{x}(1) = (1.5, 1)$.

Moments of Probability Distributions: Moments of random variables are the generalization of mean and covariance and are defined as expected values of monomials of random variables. More precisely, given $(\alpha_1, ..., \alpha_n) \in \mathbb{N}^n$ where $\alpha = \sum_{i=1}^n \alpha_i$, moment of order α of random vector ω is defined as $\mathbb{E}[\Pi_{i=1}^n \omega_i^{\alpha_i}]$. For example, sequence of the moments of order $\alpha = 2$ for $n = 3$ is defined as $\big[\mathbb{E}[w_1^2], \mathbb{E}[\omega_1\omega_2], \mathbb{E}[\omega_1\omega_3], \mathbb{E}[\omega_2^2], \mathbb{E}[\omega_2\omega_3], \mathbb{E}[\omega_3^2]\big]$. Moments of random variables can be easily computed using the characteristic function of probability distributions Jacod and Protter (2012). We will use a finite sequence of the moments to represent non-Gaussian probability distributions.

Sum of Squares Polynomials and Optimization: In this paper, we will use sum of squares (SOS) techniques to solve nonconvex optimization problems of the risk bounded trajectory planning problems. Polynomial $\mathcal{P}(x)$ is a sum of squares polynomial if it can be written as a sum of *finitely*

many squared polynomials, i.e., $\mathcal{P}(x) = \sum_{j=1}^{m} h_j(x)^2$ for some $m < \infty$ and $h_j(x) \in \mathbb{R}[x]$ for $1 \leq j \leq m$. SOS condition, i.e., $\mathcal{P}(x) \in SOS$, can be represented as a convex constraint of the form of a linear matrix inequality (LMI) in terms of the coefficients of the polynomial, i.e., $\mathcal{P}(x) \in SOS \rightarrow \mathcal{P}(x) = \mathbf{x}^T A \mathbf{x}$ where $\mathbf{x}$ is the vector of standard monomial basis and A is a positive semidefinite matrix in terms of the coefficients of the polynomial Parrilo (2003); Lasserre (2001); Laurent (2009). One can use different software packages like Yalmip Lofberg (2004) and Spotless Tobenkin et al. (2013) to check the SOS condition of the polynomials.

Sum of squares polynomials are used to obtain the convex relaxations of noncovex polynomial optimization problems Parrilo (2003); Lasserre (2001); Laurent (2009). More precisely, consider the following noncovex polynomial optimization as:

$$\begin{aligned} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & g_i(\mathbf{x}) \geq 0,\ i = 1, ..., m \end{aligned} \tag{1}$$

where f and $g_i, i = 1, ..., m$ are polynomial functions. We can rewrite the polynomial optimization in (1) as the following form:

$$\begin{aligned} &\underset{\gamma\in\mathbb{R}}{\text{maximize}}\ \gamma \\ &\text{subject to } f(\mathbf{x}) - \gamma \geq 0,\ \forall \mathbf{x} \in \{\mathbf{x} \in \mathbb{R}^n : g_i(\mathbf{x}) \geq 0\ |_{i=1}^{m}\} \end{aligned} \tag{2}$$

where we look for the best lower bound of the function $f(\mathbf{x})$ denoted by γ inside the feasible set of the original optimization problem. Hence, if $\mathbf{x}^*$ is the optimal solution of the original optimization problem in (1), then $\gamma^* = f(\mathbf{x}^*)$ is the optimal solution of the optimization problem in (2).

Note that in optimization problem (2), objective function is linear and the constraint is the nonnegativity condition of polynomial $f(\mathbf{x}) - \gamma$. Such nonnegativity condition can be replaced by SOS conditions of polynomials Parrilo (2003); Lasserre (2001); Laurent (2009). Hence, we can transform the optimization in (2) into a convex optimization, i.e., semidefinite program, with LMI constraints in terms of the coefficients of the polynomials of the original optimization in (1). Also, we can recover the optimal solution of the original polynomial optimization in (1), i.e., $\mathbf{x}^*$, using the solution of the dual convex optimization problem of (2) as shown in Lasserre (2001); Laurent (2009). One can use different software packages like GloptiPoly Henrion et al. (2009) to solve the polynomial optimization in (1) using SOS-based primal-dual approach.

Recently, SOS optimization techniques have been extended to address time-varying polynomial optimization problems of the form

$$\begin{aligned} \underset{\mathbf{x}\in\mathbb{R}^n}{\text{minimize}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & g_i(\mathbf{x}, t) \geq 0,\ \forall t \in [t_0, t_f],\ i = 1, ..., m \end{aligned} \tag{3}$$

where one needs to make sure that time-varying constraints are satisfied over the given time horizon $t \in [t_0, t_f]$. SOS-based techniques can be used to solve the time-varying optimization problem in (3) without the need for time discretization by transforming the problem into a convex optimization, i.e., time-varying semidefinite program Ahmadi and Khadir (2021); Khadir et al. (2021); Bellon et al. (2021).

3 Problem formulation

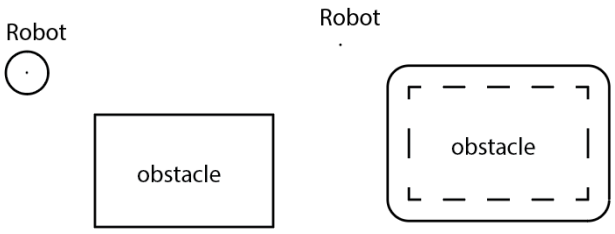


Figure 1. Left: workspace. Right: configuration space.

We make the following assumptions.

1. We mainly consider vehicle models. Most of the time the robot has 2 or 3 degrees of freedom (DOF) and lives in a 2 or 3-dimensional space. We do not consider high degrees of freedom (e.g., 5 DOF, 6 DOF, or 7 DOF) robotic arms.
2. We assume that the robot is a point. This is a reasonable assumption. Suppose in the 2-dimensional workspace, the robot can be outer-approximated by a disc with radius r. Then in the configuration space, the equivalent configuration is that the robot is a point and the obstacle's boundary is extended outwards by a distance of r (Figure 1). The 3-dimensional case is analogous. A state space usually refers to a configuration space, though sometimes can also refer to a workspace.
3. In addition to planning trajectories, we also plan tubes, which is a 2-dimensional region in 2-dimensional state space, or a 3-dimensional solid in 3-dimensional state space. One of the motivations of planning tubes is that we work directly in the workspace where the robot is outer-approximated by a disc in 2-dimensional state space or a ball in 3-dimensional state space.
4. We assume the uncertainty appeared in any problem does not necessarily have a Gaussian distribution. However, we assume that the moments of the probability distribution can be calculated and are finite.
5. We assume obstacles are represented by polynomials. They can be of arbitrary shape, can deform over time, can move, and have arbitrary uncertainty. The change in shapes and positions can be captured by polynomials parameterized by time t, and the uncertainty can be represented by random variables in the polynomial. In essence, we are considering the trajectory planning problem in the presence of "known unknowns". This is a reasonable assumption, because there are many techniques that predict the future trajectories of obstacles. For example, in the recently developed autonomous driving research community, there is a whole area devoted to predicting the future trajectories of agents, including vehicles, bicyclists, and pedestrians, in the traffic scene Gu et al. (2021); Lu et al. (2022). After getting information from the

environment via perception, the robot first predicts the future trajectories of other agents in the scene and then plans its own trajectory. The predicted trajectories can be Gaussian mixture models (GMM) Chai et al. (2020); Varadarajan et al. (2022), and they satisfy our assumption.

Suppose $\mathcal{X} \in \mathbb{R}^{n_x}$ is an uncertain environment and the sets $\mathcal{X}_{obs_i}(\omega_i) \subset \mathcal{X},\ i = 1, ..., n_{o_s}$ and $\mathcal{X}_{obs_i}(\omega_i, t) \subset \mathcal{X},\ i = n_{o_s} + 1, ..., n_{o_d}$ are the static and dynamic uncertain obstacles, respectively, where $\omega_i \in \mathbb{R}^{n_\omega},\ i = 1, ..., n_{o_s} + n_{o_d}$ are probabilistic uncertain parameters with known probability distributions. We represent static uncertain obstacles in terms of polynomials in $\mathbf{x} \in \mathcal{X}$ and uncertain parameters as follows:

$$\mathcal{X}_{obs_i}(\omega_i) = \{\mathbf{x} \in \mathcal{X} : \mathcal{P}_i(\mathbf{x}, \omega_i) \geq 0\},\ i = 1, ..., n_{o_s} \quad (4)$$

where $\mathcal{P}_i : \mathbb{R}^{n_x + n_\omega} \to \mathbb{R}, i = 1, ..., n_{o_s}$ are the given polynomials. Similarly, we represent dynamic uncertain obstacles in terms of polynomials in $\mathbf{x} \in \mathcal{X}$, time t, and uncertain parameters as follows:

$$\mathcal{X}_{obs_i}(\omega_i, t) = \{\mathbf{x} \in \mathcal{X} : \mathcal{P}_i(\mathbf{x}, \omega_i, t) \geq 0\} \,|_{i=n_{o_s}+1}^{n_{o_d}} \quad (5)$$

where $\mathcal{P}_i : \mathbb{R}^{n_x + n_\omega + 1} \to \mathbb{R}, i = n_{o_s} + 1, ..., n_{o_d}$ are the given polynomials. Note that, in general, the sets in (4) and (5) represent *nonconvex* probabilistic obstacles, e.g., nonconvex obstacles with uncertain size, location, or geometry Jasour and Williams (2019); Jasour et al. (2018).

Continuous-time trajectory planning: Given static and dynamic uncertain obstacles in (4) and (5), we define risk as the probability of collision with uncertain obstacles in the environment. In the risk bounded continuous-time trajectory planning problem, we aim at finding a continuous-time trajectory $\mathbf{x}(t) : [t_0, t_f] \to \mathbb{R}^{n_x}$ defined over the time horizon $t \in [t_0, t_f]$ between the start and final points $\mathbf{x}_0$ and $\mathbf{x}_f$ such that the probability of collision of the trajectory $\mathbf{x}(t)$ with uncertain obstacles is bounded.

More precisely, we define the risk bounded continuous-time trajectory planning problem as the following probabilistic optimization problem:

$$\underset{\mathbf{x}(t):[t_0,t_f]\to\mathbb{R}^{n_x}}{\text{minimize}} \int_{t_0}^{t_f} \|\dot{\mathbf{x}}(t)\|_2^2\, dt \quad (6)$$

$$\text{subject to} \quad \mathbf{x}(t_0) = \mathbf{x}_0,\ \mathbf{x}(t_f) = \mathbf{x}_f \quad (6a)$$

$$\text{Prob}\left(\mathbf{x}(t) \in \mathcal{X}_{obs_i}(\omega_i)\right) \leq \Delta,\ \forall t \in [t_0, t_f] \,|_{i=1}^{n_{o_s}} \quad (6b)$$

$$\text{Prob}\left(\mathbf{x}(t) \in \mathcal{X}_{obs_i}(\omega_i, t)\right) \leq \Delta,\ \forall t \in [t_0, t_f] \,|_{i=n_{o_s}+1}^{n_{o_d}} \quad (6c)$$

where objective function (6) tries to minimize the length of the trajectory. Also, constraints (6b) and (6c) are the defined risks at time t for trajectory $\mathbf{x}(t)$ in terms of the uncertain static and dynamic obstacles, respectively. Moreover, $0 \leq \Delta \in \mathbb{R} \leq 1$ is the given acceptable risk level. To solve the risk bounded optimization problem in (6), we will look for the following continuous-time trajectories:

i) Polynomial trajectories over the planning horizon $[t_0, t_f]$ of the form

$$\mathbf{x}(t) = \sum_{\alpha=0}^{d} \mathbf{c}_\alpha t^\alpha,\ t \in [t_0, t_f] \quad (7)$$

where $\mathbf{c}_\alpha \in \mathbb{R}^{n_x},\ \alpha = 0, ..., d$ are the coefficient vectors,

ii) Piecewise linear trajectories of the form

$$\mathbf{x}_i(t) = \mathbf{a}_i + \mathbf{b}_i t,\ t \in [t_{i-1}, t_i),\ i = 1, ..., s \quad (8)$$

where s is the number of linear pieces defined over the time intervals $t \in [t_{i-1}, t_i]$ of the form $t_{i-1} = t_0 + \frac{(i-1)(t_f - t_0)}{s}$ and $t_i = t_0 + \frac{i(t_f - t_0)}{s}, i = 1, ..., s$, and $\mathbf{a}_i, \mathbf{b}_i \in \mathbb{R}^{n_x}$ are the coefficient vectors.

Solving the probabilistic optimization in (6) is challenging, because i) we need to deal with multivariate integrals of the probabilistic constraints in (6b) and (6c) defined over the nonconvex sets of the obstacles, ii) we need to deal with time-varying constraints to ensure that they are all satisfied over the entire planning time horizon $[t_0, t_f]$, and iii) optimization in (6) is, in general, nonconvex optimization; Hence, we cannot guarantee to obtain the global optimal solution.

Continuous-time tube design: A tube is an n-dimensional region in the n-dimensional state space at any time. For example, while a trajectory is a 1-dimensional point moving in the 2-dimensional or 3-dimensional state space over time, a tube can be a 2-dimensional disc moving in the 2-dimensional state space or a 3-dimensional solid moving in the 3-dimensional state space over time. We build a tube around a risk bounded trajectory so that any states inside the tube are guaranteed to have bounded risk. The reasons for building tubes are three-folds:

1. When we work directly in the workspace (Figure 1 Left), the robot can be outer-approximated by a disc in the 2-dimensional state space, or by a ball in the 3-dimensional state space. We need to build tubes, which is an n-dimensional region at any time in the n-dimensional state space, instead of a point at any time as in the trajectory case, to ensure the safety of the robot. The nominal trajectory of the tube can be viewed as the trajectory of the center of the robot.
2. When we work in the configuration space (Figure 1 Right), the tube represents a collection of trajectories that are safe. If the robot is able to follow any trajectory exactly without any deviations, then the robot can follow any trajectory in the tube to reach the goal without violating any safety constraints.
3. Once we consider the dynamics of the robot, the planned continuous time trajectory can be viewed as a high level guidance for the robot to follow. Due to uncertainty in the dynamics, the robot can deviate from the nominal trajectory. However, as long as the robot stays in the tube, it is guaranteed to be safe.

A tube can be designed in many ways. For example, at any time, a tube can be an n-dimensional ball or an n-dimensional cube in the n-dimensional state space. We call such a design choice a tube parameterization. We define a tube as states near a planned trajectory $\bar{\mathbf{x}}(t)$:

$$\mathcal{Q}(\bar{\mathbf{x}}(t), t) = \{\mathbf{x} \in \mathbb{R}^{n_x} : (\mathbf{x} - \bar{\mathbf{x}}(t))^T Q(t) (\mathbf{x} - \bar{\mathbf{x}}(t)) \leq 1\} \quad (9)$$

where $Q(t)$ is a positive-definite matrix, for any $t \in [t_0, t_f]$. For example, if $Q(t)$ is a multiple of the identity matrix, then the tube is a ball centered at $\bar{\mathbf{x}}(t)$ for any t. In the

risk bounded continuous-time tube design problem, we aim to find a tube $\mathcal{Q}(\bar{\mathbf{x}}(t), t)$ associated with a continuous-time trajectory $\bar{\mathbf{x}}(t)$ over the time horizon $[t_0, t_f]$ starting from $\mathbf{x}_0$ and ending at $\mathbf{x}_f$, such that the probability of collision of any state in the tube $\mathcal{Q}(\bar{\mathbf{x}}(t), t)$ with uncertain obstacles is bounded, and we aim to maximize the size of such a tube with respect to the parameterization of the tube.

More precisely, we define the risk bounded continuous-time tube design problem as the following probabilistic optimization problem:

$$\underset{\substack{\mathbf{x}(t):[t_0,t_f]\to\mathbb{R}^{n_x}\\ Q(t)\succ 0, t\in[t_0,t_f]}}{\text{minimize}} \int_{t_0}^{t_f} \|\dot{\mathbf{x}}(t)\|_2^2\, dt + w \max_{t\in[t_0,t_f]} \lambda_{max}(Q(t)) \tag{10}$$

$$\text{subject to} \quad \mathbf{x}(t_0) = \mathbf{x}_0,\ \mathbf{x}(t_f) = \mathbf{x}_f \tag{10a}$$

$$\text{Prob}\left(\mathcal{Q}(\mathbf{x}(t), t) \cap \mathcal{X}_{obs_i}(\omega_i) \neq \emptyset\right) \leq \Delta,\ \forall t \in [t_0, t_f]\ |_{i=1}^{n_{o_s}} \tag{10b}$$

$$\text{Prob}\left(\mathcal{Q}(\mathbf{x}(t), t) \cap \mathcal{X}_{obs_i}(\omega_i, t) \neq \emptyset\right) \leq \Delta, \quad \forall t \in [t_0, t_f]\ |_{i=n_{o_s}+1}^{n_{o_d}} \tag{10c}$$

$$\mathcal{Q}(\mathbf{x}(t), t) \subseteq M(t), \forall t \in [t_0, t_f] \tag{10d}$$

where $\lambda_{max}(Q(t))$ represents the largest eigenvalue of $Q(t)$ and the second term in the objective is the maximum of all maximum eigenvalues of $Q(t)$, for each time, over the entire time horizon, weighted by a hyperparameter w. By minimizing the maximum eigenvalues of $Q(t)$, we are trying to maximize the size of the tube. For example, if $Q(t)$ is a multiple of an identity matrix for any t, then the tube is a ball for any t, and the eigenvalues of $Q(t)$ are the inverse of squared radius of the ball for any t. In this case, minimizing the maximum eigenvalues of $Q(t)$ amounts to maximizing the radius of the ball. The first term in the objective tries to minimize the trajectory length. The weight w is meant to balance maximizing the tube size and minimizing the trajectory length. Constraints (10b) and (10c) are the risks at time t for the tube $\mathcal{Q}(\mathbf{x}(t), t)$ in terms of the uncertain static and dynamic obstacles, respectively, and $0 \leq \Delta \in \mathbb{R} \leq 1$ is the given acceptable risk level. The constraint (10d) imposes an upper bound on the size of the tube. For example, if the tube is a ball for any t, then we impose an upper bound on the radius of the ball. This is reasonable, because for example we do not want the tube to grow outside the state space. While we consider polynomial and piecewise linear trajectories in trajectory planning, we consider the union of balls around the trajectory as a tube, in the form of

$$\mathcal{Q}(\bar{\mathbf{x}}(t), t) = \{\mathbf{x} \in \mathbb{R}^{n_x} : r(t)^2 - ||\mathbf{x} - \bar{\mathbf{x}}(t)||_2^2 \geq 0\} \tag{11}$$

So the constraint (10d) translates to an upper bound on the radius $r(t)$. The probabilistic optimization (10) is even more challenging than (6), because it is nonconvex and has additional terms involving the tube. The risk bound needs to hold for a whole region instead of a single trajectory.

In this paper, we provide a systematic numerical procedure to efficiently solve the probabilistic optimization problems in (6) and (10) in the presence of nonconvex uncertain obstacles with arbitrary probability distributions. For this purpose, we will leverage the notion of risk contours to transform the probabilistic optimization in (6) and (10) into deterministic polynomial optimization problems. We will use SOS optimization techniques to obtain optimal continuous-time trajectories with guaranteed bounded risk. We will also use SOS optimization and binary search to look for max-sized tube with guaranteed bounded risk.

4 Risk contours

In Jasour and Williams (2019), we define the risk contour with respect to the static uncertain obstacle in (4) and the given acceptable risk level Δ in (6b) as the set of all points in the environment whose probability of collision with the uncertain obstacle is less or equal to Δ. In this paper, we use static and dynamic risk contours defined for static and dynamic uncertain obstacles, respectively, to identify the safe regions in uncertain environments, i.e., the feasible set of optimization (6).

In Jasour and Williams (2019), to construct the risk contours of uncertain static obstacles, we propose an $(n_x + n_\omega)$-dimensional convex optimization in the form of a semidefinite program (SDP). Such optimization is not suitable for online computations and is limited to small dimensions $(n_x + n_\omega)$. In this paper, we propose an optimization-free fast approach, i.e., an analytical method, to construct the risk contours both for static and dynamic uncertain obstacles and show how one can use the obtained risk contours to solve the risk-bounded trajectory planning problem in (6).

4.1 Static Risk Contours

Let $\mathcal{X}_{obs}(\omega) = \{\mathbf{x} \in \mathcal{X} : \mathcal{P}(\mathbf{x}, \omega) \geq 0\}$ be the given static uncertain obstacle as defined in (4) and $\Delta \in [0, 1]$ be the given acceptable risk level. Then, static Δ-risk contour denoted by $\mathcal{C}_r^\Delta$ is defined as the set of all points in the environment, i.e., $\mathbf{x} \in \mathcal{X}$, whose risk is less or equal to Δ. More precisely,

$$\mathcal{C}_r^\Delta := \{\, \mathbf{x} \in \mathcal{X} :\ \text{Prob}(\mathbf{x} \in \mathcal{X}_{obs}(\omega)) \leq \Delta\} \tag{12}$$

The main idea to construct the static risk contour in (12) is to replace the probabilistic constraint, i.e., $\text{Prob}(\mathbf{x} \in \mathcal{X}_{obs}(\omega)) = \text{Prob}(\mathcal{P}(\mathbf{x}, \omega) \geq 0) \leq \Delta$, with a deterministic constraint in terms of $\mathbf{x}$. We are going to propose an analytical method to compute risk contours. For comparison purposes, we first introduce an optimization-based method, and show that it is inefficient and can be more conservative compared to our method.

4.1.1 Optimization-based Method The optimization-based method tries to find a polynomial upper bound of the indicator function associated with the risk. The problem of finding a polynomial upper bound can be formulated as an SOS programming problem.

To compute the static risk contour in (12), we obtain the deterministic constraint as follows: For a given point $\mathbf{x} \in \mathcal{X}$, the probability of collision with the uncertain static obstacle, i.e., $\text{Prob}(\mathbf{x} \in \mathcal{O}(\omega))$, is equivalent to the expectation of the indicator function of the superlevel set of $p(\mathbf{x}, \omega)$ as follows Jasour and Williams (2019); Jasour et al. (2018):

$$\text{Prob}(p(\mathbf{x},\omega)\geq 0)=\int_{\{\omega:p(\mathbf{x},\omega)\geq 0\}} pr(\omega)d\omega=\mathbb{E}[\mathbb{I}_{p\geq 0}] \tag{13}$$

where $pr(\omega)$ is the probability density function of ω and $\mathbb{I}_{p\geq 0}$ is the indicator function of the superlevel set of $p(\mathbf{x},\omega)$ defined as $\mathbb{I}_{p\geq 0}=1$ if $(\mathbf{x},\omega)\in\{(\mathbf{x},\omega):p(\mathbf{x},\omega)\geq 0\}$, and 0 otherwise. The expectation of the indicator function, however, is not necessarily easily computable. To compute the expectation value, we want to find a polynomial description of the indicator function that upper bounds the true indicator function $\mathbb{I}_{p\geq 0}$. If we can find a polynomial $P_{\mathbb{I}}:\mathbb{R}^{n_x+n_\omega}\to\mathbb{R}$ of the order d with coefficients c_{ij} that upper bounds the indicator function, i.e., $P_{\mathbb{I}}(\mathbf{x},\omega):=\sum_{(i,j)}c_{ij}x^i\omega^j\geq\mathbb{I}_{p\geq 0}$, then we can apply the expectation w.r.t. the probability density function of ω to the both sides and describe the upper bound probability in terms of the known moments of ω as follows:

$$\begin{aligned}\mathbb{E}[P_{\mathbb{I}}(\mathbf{x},\omega)]&=\sum_{(i,j)}c_{ij}x^i\mathbb{E}[\omega^j]\\&\geq\mathbb{E}[\mathbb{I}_{p\geq 0}]=\text{Prob}(p(\mathbf{x},\omega)\geq 0)\end{aligned} \tag{14}$$

where $\mathbb{E}[\omega^j]$ is the moment of order j of random vector ω. Hence, we can construct an inner approximation of the Δ-risk contour in (12), denoted by $\hat{\mathcal{C}}_r^\Delta$, using the upper bound probability in (14) as follows:

$$\hat{\mathcal{C}}_r^\Delta=\{\,\mathbf{x}\in\mathcal{X}:\ P(\mathbf{x})\leq\Delta\} \tag{15}$$

where polynomial $P(\mathbf{x})=\mathbb{E}[P_{\mathbb{I}}(\mathbf{x},\omega)]=\sum_{(i,j)}c_{ij}\mathbb{E}[\omega^j]x^i$ as shown in (14).

According to (14) and (15), the problem of constructing Δ-risk contour (12), reduces to the problem of finding an upper bound probability and an upper bound polynomial indicator function of the superlevel set of $p(\mathbf{x},\omega)$. In Jasour and Williams (2019), to compute the upper bound polynomial indicator functions, an (n_x+n_ω)-dimensional convex optimization problem in the form of a semidefinite program is provided. Such optimization is not suitable for online computations and is limited to small dimensions (n_x+n_ω).

4.1.2 Analytical Method We propose an optimization-free approach to obtain an upper bound on the probability, which is suitable for online computations and large scale problems. The key insight of this method is to use concentration inequalities to upper bound the risk, obtaining a risk contour involving equations of moments of the representing polynomials of obstacles. The computation involved is just the four basic arithmetic operations plus exponentiation.

Given the polynomial $\mathcal{P}(\mathbf{x},\omega)$ of the uncertain obstacle $\mathcal{X}_{obs}(\omega)$, we define the set $\hat{\mathcal{C}}_r^\Delta$ as follows:

$$\hat{\mathcal{C}}_r^\Delta=\left\{\mathbf{x}\in\mathcal{X}:\ \begin{array}{c}\frac{\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega)]-\mathbb{E}[\mathcal{P}(\mathbf{x},\omega)]^2}{\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega)]}\leq\Delta,\\ \mathbb{E}[\mathcal{P}(\mathbf{x},\omega)]\leq 0\end{array}\right\} \tag{16}$$

where the expectation is taken with respect to the distribution of uncertain parameter ω. Note that we can compute polynomials $\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega)]$ and $\mathbb{E}[\mathcal{P}(\mathbf{x},\omega)]$ in terms of $\mathbf{x}$ and known moments of ω. More precisely, $\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega)]$ and $\mathbb{E}[\mathcal{P}(\mathbf{x},\omega)]$ are polynomials in $\mathbf{x}$ whose coefficients are defined in terms of the moments of ω and the coefficients of polynomial $\mathcal{P}(\mathbf{x},\omega)$.

The following result holds true.

Theorem 1: The set $\hat{\mathcal{C}}_r^\Delta$ in (16) is an inner approximation of the static risk contour $\mathcal{C}_r^\Delta$ in (12).

Proof: Let $\mathcal{X}_{obs}(\omega)=\{\mathbf{x}\in\mathcal{X}:\mathcal{P}(\mathbf{x},\omega)\geq 0\}$ be the given static uncertain obstacle as defined in (4). To obtain an upper bound of the probability $\text{Prob}(\mathbf{x}\in\mathcal{X}_{obs}(\omega))=\text{Prob}(\mathcal{P}(\mathbf{x},\omega)\geq 0)$, we begin by defining a new random variable $z\in\mathbb{R}$ as follows:

$$z=\mathcal{P}(\mathbf{x},\omega) \tag{17}$$

Note that $z\in\mathbb{R}$ is a random variable, while $\mathcal{P}(\mathbf{x},\omega):\mathbb{R}^{n_x+n_\omega}\to\mathbb{R}$ is a polynomial in $\mathbf{x}\in\mathbb{R}^{n_x}$ and random vector $\omega\in\mathbb{R}^{n_\omega}$. By doing so, we can transform the (n_x+n_ω)-dimensional probability assessment problem into a one-dimensional probability assessment problem in terms of the new defined random variable z, i.e., $\text{Prob}(\mathbf{x}\in\mathcal{X}_{obs}(\omega))=\text{Prob}(\mathcal{P}(\mathbf{x},\omega)\geq 0)=\text{Prob}(z\geq 0)$, Jasour et al. (2018). Note that the statistics of the random variable z, e.g., moments, are functions of $\mathbf{x}$ and the statistics of the random vector ω.

Now, to compute an upper bound of the probability $\text{Prob}(z\geq 0)$, we just need an upper bound polynomial description of one-dimensional indicator function $\mathbb{I}_{z\geq 0}$ defined as $\mathbb{I}_{z\geq 0}=1$ if $z\geq 0$, and 0 otherwise. In this paper, we will use the upper bound polynomial indicator function and the upper bound probability provided by *Cantelli's inequality* defined for scalar random variables as

$$\text{Prob}(z\geq 0)\leq\frac{\mathbb{E}[z^2]-\mathbb{E}[z]^2}{\mathbb{E}[z^2]},\text{whenever }\mathbb{E}[z]\leq 0. \tag{18}$$

In other words, the probability of collision, i.e., $\text{Prob}(z\geq 0)=\text{Prob}(\mathcal{P}(\mathbf{x},\omega)\geq 0)$, is bounded if the expected value of remaining safe is nonnegative, i.e, $\mathbb{E}[z]=\mathbb{E}[\mathcal{P}(\mathbf{x},\omega)]\leq 0$. For other different one-dimensional indicator function-based probability bounds see Jasour et al. (2018); Jasour (2019); Nemirovski and Shapiro (2007).

Hence, the upper bound of the probability of collision can be described in terms of the polynomial of the uncertain obstacle as follows:

$$\text{Prob}(\mathbf{x}\in\mathcal{X}_{obs}(\omega))\leq\frac{\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega)]-\mathbb{E}[\mathcal{P}(\mathbf{x},\omega)]^2}{\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega)]} \tag{19}$$

whenever $\mathbb{E}[\mathcal{P}(\mathbf{x},\omega)]\leq 0$. This will results in an inner approximation of the Δ-risk contour as in (16). Note that although the standard Cantelli's inequality uses the first two moments of the scalar random variable z, we need higher order moments of random vector ω to construct the set in (16). In fact, in this paper, we generalize the standard *scalar* Cantelli probability bound to obtain multivariate probability bound (19) involving nonconvex and nonlinear sets of obstacles. ■

Remark 1: The set in (16) is a *rational* polynomial-based inner approximation of the risk contour in (12). It also uses higher order moments of the uncertain parameter ω up to order $2d$, where d is the order of the polynomial obstacle $\mathcal{P}(\mathbf{x},\omega)$. Since $\hat{\mathcal{C}}_r^\Delta$ is an inner approximation of $\mathcal{C}_r^\Delta$, any

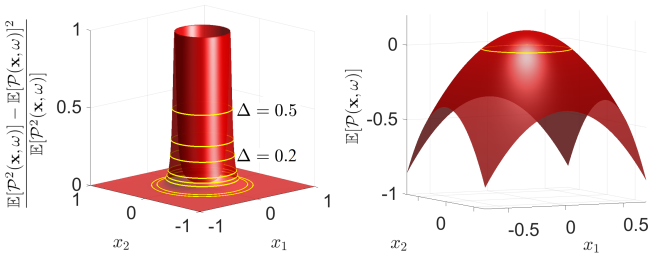


Figure 2. Illustrative example 1: Intersection of Δ-sublevel set of function $\frac{\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega)]-\mathbb{E}[\mathcal{P}(\mathbf{x},\omega)]^2}{\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega)]}$ and 0-sublevel set of function $\mathbb{E}[\mathcal{P}(\mathbf{x},\omega)]$ describes the static Δ-risk contour as in (16).

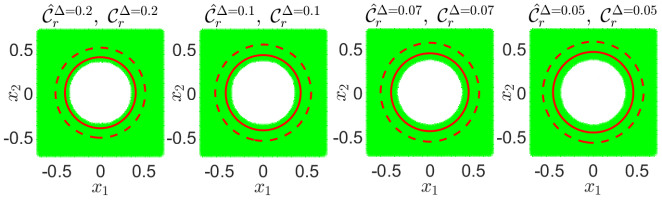


Figure 3. Illustrative example 1: True static Δ-risk contour $\mathcal{C}_r^{\Delta}$ (green) and inner approximation $\hat{\mathcal{C}}_r^{\Delta}$ obtained using i) the proposed analytical method in (16) (outside of the solid-line) and ii) the proposed optimization-based method in (Jasour and Williams (2019), Fig.4) (outside of the dashed-line). While the optimization-based method in Jasour and Williams (2019) uses a standard polynomial of order 20, the proposed analytical method in (16) uses 4th order rational and 2nd order standard polynomials to describe the risk contours.

trajectory $\mathbf{x}(t) \in \hat{\mathcal{C}}_r^{\Delta}, \forall t \in [t_0, t_f]$, is guaranteed to have a risk less or equal to Δ. □

We now provide an illustrative example to show the performance of the proposed method to construct the static Δ-risk contours and benchmark our method against the optimization-based approach in Jasour and Williams (2019).

Illustrative Example 1: Consider the following illustrative example where $\mathcal{X} = [-1, 1]^2$. The set $\mathcal{X}_{obs}(\omega) = \{(x_1, x_2) : \omega^2 - x_1^2 - x_2^2 \geq 0\}$ represents a circle-shaped obstacle whose radius ω has a uniform probability distribution over $[0.3, 0.4]$, Jasour and Williams (2019). Moment of order α of a uniform distribution defined over $[l, u]$ can be described in a closed-form as $\frac{u^{\alpha+1}-l^{\alpha+1}}{(u-l)(\alpha+1)}$. To construct the static Δ-risk contour, we compute polynomials $\mathbb{E}[\mathcal{P}(\mathbf{x}, \omega)]$ and $\mathbb{E}[\mathcal{P}^2(\mathbf{x}, \omega)]$ using the polynomial obstacle and the moments of ω as follows:

$$\begin{aligned}
\mathbb{E}[\mathcal{P}(\mathbf{x},\omega)] &= \mathbb{E}[\omega^2 - x_1^2 - x_2^2] \\
&= \mathbb{E}[\omega^2] - x_1^2 - x_2^2 = 0.1233 - x_1^2 - x_2^2 \\
\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega)] &= \mathbb{E}[\left(\omega^2 - x_1^2 - x_2^2\right)^2] \\
&= \mathbb{E}[\omega^4] - 2\mathbb{E}[\omega^2]x_1^2 - 2\mathbb{E}[\omega^2]x_2^2 \\
&\quad + x_1^4 + 2x_1^2x_2^2 + x_2^4 \\
&= 0.0156 - 0.2466x_1^2 - 0.2466x_2^2 \\
&\quad + x_1^4 + 2x_1^2x_2^2 + x_2^4
\end{aligned}$$

As shown in Figures 2 and 3, we use the sublevels of functions $\frac{\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega)]-\mathbb{E}[\mathcal{P}(\mathbf{x},\omega)]^2}{\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega)]}$ and $\mathbb{E}[\mathcal{P}(\mathbf{x}, \omega)]$ as in (16) to construct the inner approximations of the static Δ-risk contours for different risk levels $\Delta = [0.2, 0.1, 0.07, 0.05]$. We also compare our proposed method in (16) with the optimization-based method in Jasour and Williams (2019) as shown in Figure 3. Using the provided SDP in Jasour and Williams (2019), we obtain a polynomial of order 20 to describe the inner approximations of the Δ-risk contours. We note that the proposed analytical method of this paper provides a tight inner approximation of the Δ-risk contours. This is primarily due to the facts that i) the proposed analytical approach results in a *rational* polynomial representation of the risk-contours as opposed to a *standard* polynomial representation provided in Jasour and Williams (2019) and ii) with the provided analytical approach, we are able to avoid the numerical issues that arise when solving large-scale SDPs Roux et al. (2018). The proposed method of this paper is also suitable for online large scale planning problems.

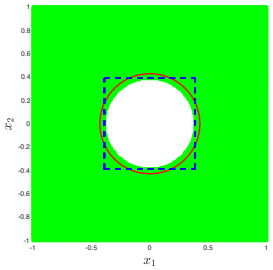


Figure 4. A simple example where $\widetilde{\mathcal{C}}_r^{\Delta}$ can be directly calculated from $\mathcal{C}_r^{\Delta}$. The green region is the true Δ-risk contour $\mathcal{C}_r^{\Delta}$, obtained from Monte Carlo simulation. The red circle is boundary of the Δ-risk contour $\hat{\mathcal{C}}_r^{\Delta}$ obtained by our analytical method. The blue dashed square is the boundary of the Δ-risk contour of $\widetilde{\mathcal{C}}_r^{\Delta}$.

4.1.3 Comparison with Linear Gaussian Method We compare our method with the linear Gaussian method proposed in Blackmore et al. (2011). In their setting, the system dynamics is discrete-time linear time invariant system with Gaussian noise. The obstacles are represented by convex polytopes. In this chapter, we compare our polynomial representation of risk contours to the convex polytopic representation of the obstacle in the linear Gaussian method.

To use the linear Gaussian method, we need to find, for each obstacle, a risk contour represented as a union of half planes

$$\widetilde{\mathcal{C}}_r^{\Delta} = \bigcup_{i=1}^{\ell} \{\mathbf{x} \in \mathcal{X} : \mathbf{a}_i^{\top}\mathbf{x} \leq b_i\}. \tag{20}$$

In principle, we can obtain $\widetilde{\mathcal{C}}_r^{\Delta}$ either from the original obstacle representation $\mathcal{C}_r^{\Delta}$ such that $\widetilde{\mathcal{C}}_r^{\Delta} \subseteq \mathcal{C}_r^{\Delta}$, or from the polynomial risk contour $\hat{\mathcal{C}}_r^{\Delta}$ obtained from the analytical method such that $\widetilde{\mathcal{C}}_r^{\Delta} \subseteq \hat{\mathcal{C}}_r^{\Delta} \subseteq \mathcal{C}_r^{\Delta}$. Given a polynomial representation of $\mathcal{C}_r^{\Delta}$ involving random variables as in (12), it is in general impossible to directly compute the half planes $\mathbf{a}_i^{\top}\mathbf{x} \leq b_i$ of $\widetilde{\mathcal{C}}_r^{\Delta}$. Nevertheless, in rare cases where the representation of $\mathcal{C}_r^{\Delta}$ is very simple, it is possible to obtain $\widetilde{\mathcal{C}}_r^{\Delta}$ directly from $\mathcal{C}_r^{\Delta}$ as shown in the simple example below.

A Simple Example. Consider Illustrative Example 1. The state space is $\mathcal{X} = [-1, 1]^2$. The set $\mathcal{O}(\omega) = \{(x_1, x_2) \in \mathcal{X} : \omega^2 - x_1^2 - x_2^2 \geq 0\}$ represents a circle-shaped obstacle whose radius ω has a uniform probability distribution

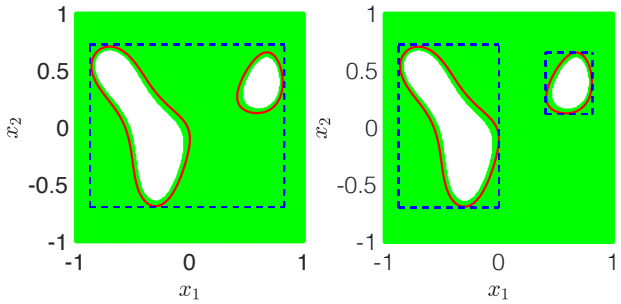


Figure 5. A more complicated example where $\widetilde{\mathcal{C}}_r^{\Delta}$ is obtained from $\hat{\mathcal{C}}_r^{\Delta}$. The green region is the true Δ-risk contour $\mathcal{C}_r^{\Delta}$, obtained from Monte Carlo simulation. The red curves are the boundary of the Δ-risk contour $\hat{\mathcal{C}}_r^{\Delta}$ obtained by our analytical method. The blue dashed rectangle is the boundary of the Δ-risk contour of $\widetilde{\mathcal{C}}_r^{\Delta}$. Left: One-rectangle approximation. Right: Two-rectangle approximation.

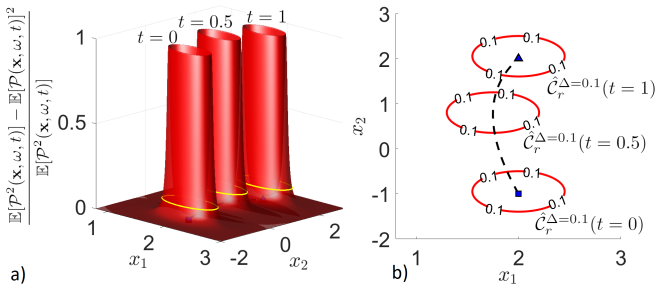


Figure 6. Illustrative example 2: a) Function $\frac{\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega,t)]-\mathbb{E}[\mathcal{P}(\mathbf{x},\omega,t)]^2}{\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega,t)]}$ at time steps $t=0,0.5,1$, b) Dynamic Δ-risk contours for $\Delta=0.1$ at time steps $t=0,0.5,1$ described in (22). Dashed line shows the expected value of the given uncertain trajectory, i.e., $\mathbb{E}[(p_{x_1}(t,\omega_2),p_{x_2}(t,\omega_3))]$. At each time t, for any point inside $\hat{\mathcal{C}}_r^{\Delta}(t)$ (outside of the closed curve), probability of collision with the moving uncertain obstacle is less or equal to $\Delta=0.1$.

over $[0.3,0.4]$. Suppose we want to use a rectangle whose sides are parallel to the axis as the boundary of $\widetilde{\mathcal{C}}_r^{\Delta}$. The square $S(\omega)=\{(x_1,x_2)\in\mathcal{X}:-\omega\le x_1,x_2\le\omega\}$ is an outer approximation of the obstacle $\mathcal{O}(\omega)$ for any $\omega\in[0.3,0.4]$. So $\mathcal{X}\backslash S(\omega)=\{(x_1,x_2)\in\mathcal{X}:x_1<-\omega\}\cup\{(x_1,x_2)\in\mathcal{X}:x_2<-\omega\}\cup\{(x_1,x_2)\in\mathcal{X}:x_1>\omega\}\cup\{(x_1,x_2)\in\mathcal{X}:x_2>\omega\}$. Suppose $\Delta=0.1$, then $\widetilde{\mathcal{C}}_r^{\Delta}$ can be calculated from $\mathcal{X}\backslash S(\omega)$ as $\{(x_1,x_2)\in\mathcal{X}:x_1<-0.39\}\cup\{(x_1,x_2)\in\mathcal{X}:x_2<-0.39\}\cup\{(x_1,x_2)\in\mathcal{X}:x_1>0.39\}\cup\{(x_1,x_2)\in\mathcal{X}:x_2>0.39\}$. The boundary of $\widetilde{\mathcal{C}}_r^{\Delta}$ is plotted as a dashed square in Figure 4.

Apart from the simple example above, $\widetilde{\mathcal{C}}_r^{\Delta}$ can rarely be directly calculated from $\mathcal{C}_r^{\Delta}$. In those non-trivial cases, we have to obtain $\hat{\mathcal{C}}_r^{\Delta}$ first and then inner approximate it to get $\widetilde{\mathcal{C}}_r^{\Delta}$. The resulting approximation can be very conservative as shown in the example below.

A More Complicated Example. Consider Example 7.1.1, where the uncertain obstacle is represented by a 5th-order polynomial and the noise has Beta distribution. Suppose we want to use a rectangle whose sides are parallel to the axis as the boundary of $\widetilde{\mathcal{C}}_r^{\Delta}$. As shown in Figure 5 Left, the boundary of $\widetilde{\mathcal{C}}_r^{\Delta}$ is plotted in dashed lines and it is so conservative that it excludes the feasible region between two components of the obstacle. In order to be less conservative, we use two rectangles to approximate the feasible region as shown in Figure 5 Right.

In summary, given a polynomial representation of an obstacle, the risk contour of the linear Gaussian method is in general more conservative than that of our method. Since in non-trivial cases, we have to obtain $\hat{\mathcal{C}}_r^{\Delta}$ first and then inner approximate it to get $\widetilde{\mathcal{C}}_r^{\Delta}$, the linear Gaussian method always takes more computational time. In very simple and rare cases, the risk contour of the linear Gaussian method can be calculated directly from the obstacle representation and is not too conservative.

4.2 Dynamic Risk Contours

Let $\mathcal{X}_{obs}(\omega,t)=\{\mathbf{x}\in\mathcal{X}:\mathcal{P}(\mathbf{x},\omega,t)\ge 0\}$ be the given dynamic uncertain obstacle as defined in (5) and $\Delta\in[0,1]$ be the given acceptable risk level in (6c). Then, dynamic Δ-risk contour at time t denoted by $\mathcal{C}_r^{\Delta}(t)$ is defined as the set of all points in the environment, i.e., $\mathbf{x}\in\mathcal{X}$, whose risk at time t is less or equal to Δ. More precisely,

$$\mathcal{C}_r^{\Delta}(t):=\{\,\mathbf{x}\in\mathcal{X}:\ \text{Prob}(\mathbf{x}\in\mathcal{X}_{obs}(\omega,t))\le\Delta\} \tag{21}$$

Similar to the static risk contours, we can replace the probabilistic constraint in (21) with deterministic constraints and construct an inner approximation of the dynamic Δ-risk contour denoted by $\hat{\mathcal{C}}_r^{\Delta}(t)$ as follows:

$$\hat{\mathcal{C}}_r^{\Delta}(t)=\left\{\mathbf{x}\in\mathcal{X}:\ \begin{array}{c}\frac{\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega,t)]-\mathbb{E}[\mathcal{P}(\mathbf{x},\omega,t)]^2}{\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega,t)]}\le\Delta,\\ \mathbb{E}[\mathcal{P}(\mathbf{x},\omega,t)]\le 0\end{array}\right\} \tag{22}$$

Note that dynamic Δ-risk contour (22) is described in terms of the time-varying constraints.

Illustrative Example 2: Consider the following illustrative example where $\mathcal{X}=[-1,1]^2$. The set $\mathcal{X}_{obs}(\omega,t)=\{(x_1,x_2):\omega_1^2-(x_1-p_{x_1}(t,\omega_2))^2-(x_2-p_{x_2}(t,\omega_3))^2\ge 0\}$ represents a moving circle-shaped obstacle with uncertain radius ω_1 and uncertain trajectories $p_{x_1}(t,\omega_2)=2-t+t^2+0.2\omega_2$, $p_{x_2}(t,\omega_3)=-1+4t-t^2+0.1\omega_3$ that describe the uncertain motion of the obstacle over the time horizon $t\in[0,1]$. Uncertain parameters have uniform, normal, and Beta distributions as $\omega_1\sim Uniform[0.3,0.4]$, $\omega_2\sim\mathcal{N}(0,0.1)$, $\omega_3\sim Beta(3,3)$. Moment of order α of a Beta distribution with parameters (a,b) and a normal distribution with mean μ and standard deviation σ can be described in closed-forms as $y_\alpha=\frac{a+\alpha-1}{a+b+\alpha-1}y_{\alpha-1}, y_0=1$ and $y_\alpha=\sigma^\alpha(-\sqrt{-1}\sqrt{2})^\alpha kummerU(\frac{-\alpha}{2},\frac{1}{2},\frac{-\mu^2}{2\sigma^2})$, respectively, where $kummerU$ is Kummer's (confluent hyper-geometric) function Kummer (1837). Similar to illustrative example 1, we compute the polynomials $\mathbb{E}[\mathcal{P}^2(\mathbf{x},\omega,t)]$ and $\mathbb{E}[\mathcal{P}(\mathbf{x},\omega,t)]$ using the moments of the uncertain parameters $\omega_i, i=1,2,3$ and the polynomial obstacle. We then construct the dynamic Δ-risk contour as a function of time as described in (22). Figure 6 shows the obtained dynamic Δ-risk contours for $\Delta=0.1$ at time steps $t=0,0.5,1$ along the given uncertain trajectory $(p_{x_1}(t,\omega_2),\ p_{x_2}(t,\omega_3))$.

Remark 2: We can use (16) and (22) to construct static and dynamic risk contours in real-time. Therefore, standard

motion planning algorithms such as RRT*, PRM, and motion primitive-based methods can be used for real-time risk bounded motion planning. To accomplish this, one just needs to use the safe regions, i.e, risk contours, to construct the trajectories. □

In the next section, we provide *continuous-time* planning algorithms to look for trajectories with guaranteed bounded risk over the entire planning time horizon without the need for *time discretization*.

5 Continuous-time risk bounded trajectory planning using risk contours

In this section, we will use the static and dynamic risk contours to solve the continuous-time risk bounded trajectory planning problem defined in (6). More precisely, we use the static and dynamic risk contours to transform the probabilistic optimization in (6) into a deterministic polynomial optimization. The obtained deterministic polynomial optimization is, in general, nonconvex and nonlinear. In addition, we need to ensure that all the obtained deterministic constraints are satisfied over the entire planning time horizon $[t_0, t_f]$. In this section, we provide convex methods based on SOS techniques introduced in Section 2 to efficiently solve the obtained nonconvex time-varying deterministic planning optimization problem. While the existing planners rely on time discretization to verify the planning safety constraints, the provided SOS-based planners look for continuous-time trajectories with guaranteed bounded risk over the entire planning time horizon without the need for time discretization.

We first begin by addressing the continuous-time risk bounded trajectory planning in static uncertain environments using the static risk contours. We then use the dynamic risk contours to address the planning problems in dynamic uncertain environments.

5.1 Planning in Static Uncertain Environments

In this section, we are concerned with continuous-time risk bounded trajectory planning in the presence of static uncertain obstacles of the form (4). More precisely, we aim at solving the probabilistic optimization problem in (6) in the presence of static uncertain obstacles $\mathcal{X}_{obs_i}(\omega_i), i = 1, ..., n_{o_s}$, i.e.,

$$\underset{\mathbf{x}(t):[t_0,t_f]\to\mathbb{R}^{n_x}}{\text{minimize}} \int_{t_0}^{t_f} \|\dot{\mathbf{x}}(t)\|_2^2 \, dt \tag{23}$$

$$\text{subject to} \quad \mathbf{x}(t_0) = \mathbf{x}_0, \ \mathbf{x}(t_f) = \mathbf{x}_f \tag{23a}$$

$$\text{Prob}\left(\mathbf{x}(t) \in \mathcal{X}_{obs_i}(\omega_i)\right) \leq \Delta, \ \forall t \in [t_0, t_f], \ i = 1, ..., n_{o_s} \tag{23b}$$

To obtain the deterministic polynomial optimization, we replace probabilistic constraints (23b) with deterministic constraints in terms of the static Δ-risk contours as follows:

$$\underset{\mathbf{x}(t):[t_0,t_f]\to\mathbb{R}^{n_x}}{\text{minimize}} \int_{t_0}^{t_f} \|\dot{\mathbf{x}}(t)\|_2^2 \, dt \tag{24}$$

$$\text{subject to} \quad \mathbf{x}(t_0) = \mathbf{x}_0, \ \mathbf{x}(t_f) = \mathbf{x}_f \tag{24a}$$

$$\mathbf{x}(t) \in \hat{\mathcal{C}}_{r_i}^{\Delta}, \ \forall t \in [t_0, t_f], \ i = 1, ..., n_{o_s} \tag{24b}$$

where $\hat{\mathcal{C}}_{r_i}^{\Delta}$ is the static Δ-risk contour of the static uncertain obstacle $\mathcal{X}_{obs_i}(\omega_i)$. The set of $\hat{\mathcal{C}}_{r_i}^{\Delta}, i = 1, ..., n_{o_s}$ represents the inner approximation of the feasible set of the probabilistic optimization in (23). The obtained deterministic optimization in (24) is time-varying optimization problem where we need to ensure to satisfy the constraints over the entire planning time horizon $[t_0, t_f]$.

To solve the deterministic optimization in (24), we will look for i) polynomial trajectory defined in (7) and ii) piece-wise linear trajectory defined in (8). By substituting the polynomial trajectory $\mathbf{x}(t) = \sum_{\alpha=0}^{d} \mathbf{c}_\alpha t^\alpha$ in optimization (24), we obtain a deterministic optimization with constraints in terms of the coefficients of the polynomial trajectory $\mathbf{x}(t)$, i.e., $(1-\Delta)\mathbb{E}[\mathcal{P}^2(\sum_{\alpha=0}^{d} \mathbf{c}_\alpha t^\alpha, \omega)] - \mathbb{E}[\mathcal{P}(\sum_{\alpha=0}^{d} \mathbf{c}_\alpha t^\alpha, \omega)]^2 \leq 0$, $\mathbb{E}[\mathcal{P}(\sum_{\alpha=0}^{d} \mathbf{c}_\alpha t^\alpha, \omega)] \leq 0$. Similarly, by substituting the piece-wise linear trajectory of (8), we obtain the objective function $\sum_{j=1}^{s} \int_{t_{j-1}}^{t_j} \|\dot{\mathbf{x}}_j(t)\|_2^2 \, dt$ and constraints $\mathbf{a}_j + \mathbf{b}_j t \in \hat{\mathcal{C}}_{r_i}^{\Delta}, \ \forall t \in [t_{j-1}, t_j], \ i = 1, ..., n_{o_s}, j = 1, ..., s$. To solve the obtained time-varying deterministic polynomial optimization problems, we will provide 3 methods based on SOS optimization techniques as follows:

5.1.1 Time-Varying SOS Optimization we use time-varying SOS optimization, introduced in Section 2, to solve the time-varying deterministic polynomial optimization in (24). In implementation, we use the heuristic algorithm introduced by Khadir et al. (2021).

5.1.2 Standard SOS Optimization In this method, we obtain a standard polynomial optimization of the form (1) by eliminating time t. We then use the standard SOS optimization technique in (2) to solve the obtained optimization problem. To achieve this, let $\mathcal{X}_{obs}(\omega) = \{\mathbf{x} \in \mathcal{X} : \mathcal{P}(\mathbf{x}, \omega) \geq 0\}$ be the given static uncertain obstacle defined in (4) and $\mathbf{x}(t)$ be the polynomial trajectory in (7). To eliminate time t, instead of using the instant risk as in (23b), we work with the average risk defined as $\frac{1}{t_f - t_0} \int \int_{\{(t,\omega):\mathcal{P}(\mathbf{x}(t),\omega)\geq 0\}} pr(\omega) d\omega dt$ where $pr(\omega)$ is the probability density function of ω. This is equivalent to treating t as a random variable with a uniform probability distribution over the planning time horizon $[t_0, t_f]$. We should note that the average risk is a *weak* safety measure, which means that just by bounding the average risk, we cannot guarantee to satisfy the constraints of the probabilistic optimization in (23).

By defining the average risk, we substitute the trajectory $\mathbf{x}(t)$ in the probabilistic constraint and follow the same steps as in Section 4.1 to obtain an upper bound of the average risk and construct the set of all coefficients $\mathbf{c}_\alpha|_{\alpha=0}^{d}$ that results in a risk bounded trajectory of the form $\mathbf{x}(t) = \sum_{\alpha=0}^{d} \mathbf{c}_\alpha t^\alpha$. More precisely, we obtain the following set for the coefficients:

$$\left\{ \mathbf{c}_\alpha|_{\alpha=0}^{d} : \begin{array}{c} (1-\Delta)\mathbb{E}[\mathcal{P}^2(\sum_{\alpha=0}^{d} \mathbf{c}_\alpha t^\alpha, \omega)] - \mathbb{E}[\mathcal{P}(\sum_{\alpha=0}^{d} \mathbf{c}_\alpha t^\alpha, \omega)]^2 \leq 0, \\ \mathbb{E}[\mathcal{P}(\sum_{\alpha=0}^{d} \mathbf{c}_\alpha t^\alpha, \omega)] \leq 0 \end{array} \right\}$$

where the expectation is taken with respect to the probability distributions of ω and t. We can obtain a similar set for the coefficients of the piece-wise linear trajectories, as well. By computing such sets for coefficients of the trajectories, we can transform the probabilistic optimization problem into a

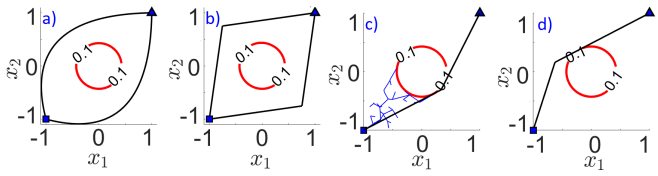


Figure 7. Illustrative Example 3: a) risk bounded polynomial trajectories of order 2 obtained via standard SOS optimization, b) risk bounded piece-wise linear trajectories obtained via standard SOS optimization, c) risk bounded piece-wise linear trajectory obtained via RRT-SOS algorithm, d) risk bounded piece-wise linear trajectory obtained via time-varying SOS optimization.

deterministic standard polynomial optimization and use the standard SOS optimization technique in (2) to obtain the optimal solution.

5.1.3 RRT-SOS In this method, we use sampling-based motion planning algorithms like RRT to construct the risk bounded trajectory of the deterministic polynomial optimization in (24). To ensure safety along the edges of the RRT, we use an SOS-based continuous-time technique to verify the constraints of the optimization in (24b) as follows:

Let $\mathbf{x}(t) = \sum_{\alpha=0}^{d} \mathbf{c}_\alpha t^\alpha, t \in [t_1, t_2]$ be the given trajectory between the two samples $\mathbf{x}_1 \in \mathcal{X}$ and $\mathbf{x}_2 \in \mathcal{X}$ in the uncertain environment. Also, let $\mathcal{S} = \{\mathbf{x} : g_i(x) \geq 0, i = 1, ..., \ell\}$ be the feasible set of optimization (24), i.e., the set constructed by the polynomial constraints of all the risk contours $\hat{\mathcal{C}}^{\Delta}_{r_i}, i = 1, ..., n_{o_s}$. Then, the following result holds true:

Polynomial $\mathbf{x}(t)$ satisfies the safety constraints of the deterministic optimization in (24) over the time interval $[t_1, t_2]$, i.e., $\mathbf{x}(t) \in \mathcal{S}$ for all $t \in [t_1, t_2]$, if and only if polynomials $g_i(\mathbf{x}(t)), i = 1, ..., \ell$ take the following SOS representation:

$$g_i(\mathbf{x}(t)) = \sigma_{0i}(t) + \sigma_{1i}(t)(t - t_1) + \sigma_{2i}(t)(t_2 - t) \;|_{i=1}^{\ell} \tag{25}$$

where $\sigma_{0i}(t), \sigma_{1i}(t), \sigma_{2i}(t), i = 1, ..., \ell$ are SOS polynomials with appropriate degrees Putinar (1993); Lasserre (2001); Laurent (2009). Yalmip Lofberg (2004) and Spotless Tobenkin et al. (2013) packages can be used to check the SOS condition (25) for the given trajectory $\mathbf{x}(t)$.

Remark 3: The complexity of the safety SOS condition in (25) is independent of the size of the planning time horizon $[t_1, t_2]$ and the length of the polynomial trajectory $\mathbf{x}(t)$. Hence, one can use (25) to easily verify the safety of trajectories in uncertain environments over the long planning time horizons. □

The safety condition in (25) can be used in any sampling-based motion planning algorithms to verify the safety of the trajectories between the sample points. In this paper, we will use the following naive RRT algorithm: To expand the RRT, we use a linear trajectory to connect the given sample point to the closest vertex in the tree if the linear trajectory satisfies the SOS condition in (25), i.e., this implies that the linear trajectory between the two points is inside the risk contours in (24b). As a termination condition, we also check the safety of the linear trajectory between the selected safe sample and the goal points via the SOS condition in (25). If the linear trajectory satisfies SOS condition (25), we connect the sample point to the goal point; Hence, a feasible trajectory between the start and goal points can be constructed.

To improve the obtained feasible trajectory, we perform the following steps: i) given the obtained tree, we construct a graph, e.g., PRM, whose nodes are the vertex of the tree and edges of the graph are all the linear trajectories between the nodes that satisfy the SOS condition in (25), ii) we then perform a shortest path algorithm, e.g., Dijkstra algorithm, to obtain a path from the start to the goal point. We can also use smart initialization to guide the RRT-SOS algorithm and improve the run-time. For example, we use the straight line between the start and goal points to initialize the RRT-SOS algorithm. We then perform the sampling in the neighborhood of the given initial path and incrementally increase the size of the neighborhood until a feasible trajectory is obtained.

Illustrative Example 3: Consider the uncertain obstacle in illustrative example 1. We want to find a risk bounded trajectory between the points $\mathbf{x}(0) = [-1, -1]$ and $\mathbf{x}(1) = [1, 1]$ by solving the probabilistic optimization in (23) with $\Delta = 0.1$. For this purpose, we solve the deterministic optimization problem in (24) with respect to the 0.1-risk contour of the uncertain obstacle using the discussed 3 methods as shown in Figure 7. More precisely, we use standard SOS optimization to obtain a polynomial trajectory of order 2 and also a piece-wise linear trajectory with 2 pieces. Using GloptiPoly package, we extract two risk bounded trajectories between the given start and goal points as shown in Figure 7-a and 7-b. We also use RRT-SOS algorithm and time-varying SOS optimization to obtain piece-wise linear trajectories as shown in Figure 7-c and 7-d, respectively. We note that the standard SOS optimization-based method results in conservative trajectories as shown in Figure 7-a and 7-b.

We compare our proposed methods with Monte Carlo-based risk bounded RRT algorithm that uses uncertainty samples and time discretization to look for the risk bounded trajectories. To verify the safety of the edges in the RRT, we use only 10 uncertainty samples and 20 uniformly sampled way-points on the edges. Such RRT algorithm is significantly slower and also does not provide any guaranteed risk bounded trajectories.

The run-time for the standard SOS optimization to obtain the piece-wise linear and polynomial trajectories are both roughly 1.5 seconds. Also, the run-time for the time-varying SOS optimization is roughly 2.2 seconds (for more information see Section 7.5). The run-time for the RRT-SOS algorithm to obtain feasible and optimal trajectories are roughly 9.3 and 92.1 seconds, respectively. The run-time for the Monte Carlo-based risk bounded RRT algorithm to obtain feasible and optimal trajectories are roughly 215.12 and 3471.4 seconds, respectively. Also, continuous-time safety verification of each edge in the RRT-SOS algorithm via (25) takes roughly 0.1 seconds while the sampling-based safety verification in the Monte Carlo-based risk bounded RRT algorithm takes roughly 3 seconds.

5.1.4 Comparison with Linear Gaussian Method We compare the RRT-SOS algorithm with the linear Gaussian

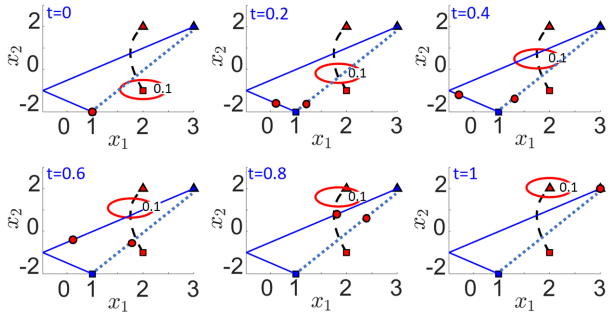


Figure 8. Illustrative Example 4: Risk bounded trajectories between the start (square) and goal (triangle) points obtained via the time-varying SOS optimization (solid line) and RRT-SOS algorithm (dashed line) in the presence of the probabilistic moving obstacle.

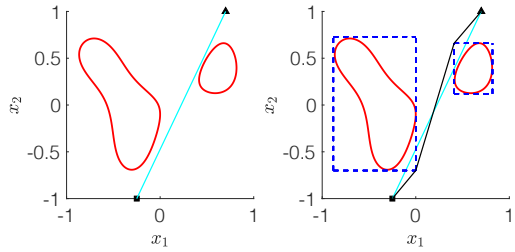


Figure 9. Comparison between RRT-SOS and the modified linear Gaussian method. The square dot represents the initial state and the triangle dot represents the goal state. RRT-SOS is using the risk contour whose boundary is represented by red curves. The cyan line between the initial and the goal states is the path planned by RRT-SOS. In contrast, the blue dotted rectangles are the boundary of the risk contour used by the linear Gaussian method, and the black line segments is the path planned by the linear Gaussian method.

method proposed in Blackmore et al. (2011). In their setting, the system dynamics is discrete-time linear time invariant system with Gaussian noise. The obstacles are represented by convex polytopes. In Section 4.1.3, we have compared the risk contour of the linear Gaussian method with our method. Since the trajectory planning problem does not consider system dynamics, we modify the linear Gaussian method to exclude the system dynamics. The modified linear Gaussian method is to solve the following optimization problem:

$$\underset{\mathbf{x}(t):[t_0,t_f]\to\mathbb{R}^{n_x}}{\text{minimize}} \int_{t_0}^{t_f} \|\dot{\mathbf{x}}(t)\|_2^2\, dt \tag{26}$$

$$\text{subject to} \quad \mathbf{x}(t_0)=\mathbf{x}_0,\ \mathbf{x}(t_f)=\mathbf{x}_f \tag{26a}$$

$$\mathbf{x}(t)\in \widetilde{\mathcal{C}}_{r_i}^{\Delta},\ \forall t\in[t_0,t_f],\ i=1,...,n_{o_s} \tag{26b}$$

Since for each obstacle a risk contour is represented as a union of half planes

$$\widetilde{\mathcal{C}}_{r}^{\Delta}=\bigcup_{i=1}^{\ell}\{\mathbf{x}\in\mathcal{X}:\mathbf{a}_i^\top\mathbf{x}\le b_i\}, \tag{20}$$

if we look for piecewise linear trajectories (8), then the optimization problem (26) becomes a disjunctive programming problem, which can also be formulated as a mixed integer programming problem.

Consider the example in Section 7.1.1, the 2D uncertain obstacle represented by a 5th order polynomial. We want to plan a trajectory from the initial state $(-0.25,-1)$ to the goal state $(0.7,1)$ with bounded risk of 0.1. Using the risk contour obtained from our analytical method, the RRT-SOS algorithm finds the risk bounded path as a straight line between the initial state and the goal state, plotted as a cyan line in Figure 9. The linear Gaussian method uses a more conservative obstacle representation and finds the piecewise linear path consisting of 3 line segments, plotted as black line segments in Figure 9.

The advantages of our method include: i) Our method utilizes a less conservative risk contour. ii) Our method is more efficient. Our method is solving convex programming programs in building the RRT tree, while the linear Gaussian method is solving a disjunctive programming problem and needs to manually try out certain parameters such as the number of waypoints in the trajectory. When the number of obstacles is large, the disjunctive programming problem becomes hard to solve, while our method checks one obstacle at a time using a convex program and hence the complexity of the convex program is not affected by the number of obstacles. iii) Our method provides safety guarantees for the entire continuous-time trajectory, while the linear Gaussian method does not.

5.2 Planning in Dynamic Uncertain Environments

In this section, we are concerned with continuous-time risk bounded trajectory planning in the presence of dynamic uncertain obstacles of the form (5). More precisely, we aim at solving the optimization problem in (6) in the presence of dynamic uncertain obstacles $\mathcal{X}_{obs_i}(\omega_i,t), i=1,...,n_{o_d}$, i.e.,

$$\underset{\mathbf{x}(t):[t_0,t_f]\to\mathbb{R}^{n_x}}{\text{minimize}} \int_{t_0}^{t_f} \|\dot{\mathbf{x}}(t)\|_2^2\, dt \tag{27}$$

$$\text{subject to} \quad \mathbf{x}(t_0)=\mathbf{x}_0,\ \mathbf{x}(t_f)=\mathbf{x}_f \tag{27a}$$

$$\text{Prob}\left(\mathbf{x}(t)\in\mathcal{X}_{obs_i}(\omega_i,t)\right)\le\Delta,\ \forall t\in[t_0,t_f]\ |_{i=n_{o_s}+1}^{n_{o_d}} \tag{27b}$$

To obtain the deterministic optimization, we replace probabilistic constraints (27b) with deterministic constraints in terms of the dynamic Δ-risk contours as follows:

$$\underset{\mathbf{x}(t):[t_0,t_f]\to\mathbb{R}^{n_x}}{\text{minimize}} \int_{t_0}^{t_f} \|\dot{\mathbf{x}}(t)\|_2^2\, dt \tag{28}$$

$$\text{subject to} \quad \mathbf{x}(t_0)=\mathbf{x}_0,\ \mathbf{x}(t_f)=\mathbf{x}_f \tag{28a}$$

$$\mathbf{x}(t)\in\hat{\mathcal{C}}_{r_i}^{\Delta}(t),\ \forall t\in[t_0,t_f],\ i=n_{o_s}+1,...,n_{o_d} \tag{28b}$$

where $\hat{\mathcal{C}}_{r_i}^{\Delta}(t)$ is the dynamic Δ-risk contour of the dynamic uncertain obstacle $\mathcal{X}_{obs_i}(\omega_i,t)$. The set of $\hat{\mathcal{C}}_{r_i}^{\Delta}(t), i=o_{n_s}+1,...,o_{n_d}$ represents the inner approximation of the feasible set of the probabilistic optimization in (27). In the deterministic polynomial optimization of (28), we need to make sure that the constraints are satisfied over the entire planning time horizon $[t_0,t_f]$. To solve the time-varying deterministic polynomial optimization in (28), we will use i) time-varying SOS optimization described in Section 5.1.1 and also ii) sampling-based motion planning algorithm similar to the one in Section 5.1.3 that uses the SOS-based

continuous-time safety verification. In the sampling-based algorithm, given the dynamic nature of the environment, one needs to verify the safety of the trajectory between the given two samples only for the time interval that is needed to traverse between the points. In this paper, we will use the following naive RRT-SOS algorithm: RRT algorithm looks for piece-wise linear trajectories with s number of linear pieces defined over the time intervals $\Delta t_i = [t_{i-1}, t_i), i = 1, ..., s$ as in (8). To construct the tree, we first fix the number of linear pieces s and build the tree for each time interval incrementally. To expand the tree for the time interval Δt_i, we connect the given sample point to a vertex in the tree, built for the time interval Δt_{i-1}, if the linear trajectory between the two points satisfies SOS condition (25) for the time interval Δt_i. Moreover, to construct the tree for the time interval Δt_{s-1}, we verify the safety of the linear trajectory between the given sample and the goal point for the time interval Δt_s as well.

Illustrative Example 4: Consider the uncertain moving obstacle in illustrative example 2. We want to find a risk bounded trajectory between the points $\mathbf{x}(0) = [1, -2]$ and $\mathbf{x}(1) = [3, 2]$ by solving the probabilistic optimization in (27) with $\Delta = 0.1$. For this purpose, we solve the deterministic optimization problem in (28) with respect to the dynamic 0.1-risk contour of the uncertain obstacle via the time-varying SOS optimization and RRT-SOS algorithm as shown in Figure 8. Note that although the obtained RRT-SOS trajectory looks like a straight line, it is a piece-wise linear trajectory consisting of 2 pieces with different velocities. The run-time for the time-varying SOS optimization and RRT-SOS algorithm are roughly 6.9 and 0.5 seconds, respectively.

6 Continuous-time risk bounded tube design using risk contours

In this section, we extend planning continuous-time risk bounded trajectories to planning continuous-time risk bounded tubes in the presence of both static and dynamic uncertain obstacles. As in trajectory planning, we convert the probabilistic constraints into deterministic constraints as follows:

$$\underset{\substack{\mathbf{x}(t):[t_0,t_f]\to\mathbb{R}^{n_x}\\ Q(t)\succ 0, t\in[t_0,t_f]}}{\text{minimize}} \int_{t_0}^{t_f} \|\dot{\mathbf{x}}(t)\|_2^2\, dt + w \max_{t\in[t_0,t_f]} \lambda_{max}(Q(t)) \tag{29}$$

$$\text{subject to} \quad \mathbf{x}(t_0) = \mathbf{x}_0,\ \mathbf{x}(t_f) = \mathbf{x}_f \tag{29a}$$

$$\mathcal{Q}(\mathbf{x}(t), t) \subseteq \hat{\mathcal{C}}_{r_i}^{\Delta},\ \forall t \in [t_0, t_f]\ |_{i=1}^{n_{o_s}} \tag{29b}$$

$$\mathcal{Q}(\mathbf{x}(t), t) \subseteq \hat{\mathcal{C}}_{r_i}^{\Delta}(t),\ \forall t \in [t_0, t_f]\ |_{i=n_{o_s}+1}^{n_{o_d}} \tag{29c}$$

$$\mathcal{Q}(\mathbf{x}(t), t) \subseteq M(t), \forall t \in [t_0, t_f] \tag{29d}$$

where $\hat{\mathcal{C}}_{r_i}^{\Delta}$ and $\hat{\mathcal{C}}_{r_i}^{\Delta}(t)$ are Δ-risk contours of static and dynamic obstacles, representing inner approximations of the feasible set of the original probabilistic optimization in (10).

There are two tube design choices. **Method (1)**: The first choice is that we use the methods in the previous section to plan a polynomial trajectory or a piecewise linear trajectory, $\bar{\mathbf{x}}(t)$. Then we build a max-sized tube $\mathcal{Q}(\bar{\mathbf{x}}(t), t)$ along this trajectory.

To search for the max-sized tube with respect to parameterization between two points, we use Algorithm 1, a binary search algorithm that uses verification of tube safety as a sub-procedure. The algorithm searches in a one-dimensional parameter space that controls the size of the tube. If the size of the tube is actually controlled by multiple parameters, the other parameters can be temporarily fixed, while one parameter is being searched. The input to the algorithm is the trajectory $\mathbf{x}(t)$, the interval I_t for the parameter t, the upperbound c_{max} on the parameter that controls the tube size, and the tolerance ε within which the difference between the output of the algorithm and the true max tube size should lie. The parameter c to be searched for has a lowerbound $l = 0$ and an upperbound $r = c_{max}$. The Verify$(\mathbf{x}(t), I_t, c)$ function takes in the trajectory $\mathbf{x}(t)$, the interval I_t, and the current tube parameter c. It returns True if the tube with parameter c is verified safe, and False if the tube is not verified to be safe. Every iteration, we consider the middle point m of the interval $[l, r]$. If the tube with parameter m is verified to be safe, then the optimal c is in the interval $[m, r]$, otherwise it is in the interval $[l, m]$. Once the length of the search interval $[l, r]$ is within ε, the algorithm output the obtained parameter $c_{out} = l$. Since the optimal parameter c_{opt} is in $[l, r]$, the difference between c_{out} and c_{opt} is $\leq \varepsilon$.

Algorithm 1 Max-sized tube algorithm

```
procedure BUILDTUBE(x(t), I_t, c_max, ε)
    l ← 0
    r ← c_max
    if Verify(x(t), I_t, l) then
        while r − l > ε do
            m ← (l + r)/2
            if Verify(x(t), I_t, m) then
                l ← m
            else
                r ← m
            end if
        end while
        Return Size = l
    else
        Return FAIL
    end if
end procedure
```

The most important sub-procedure for the algorithm is to verify if the tube with parameter c is safe. As in (25), we can verify the safety of the tube using SOS programming Jasour et al. (2021b). Let $\mathcal{S}(t) = \{\mathbf{x} : g_i(\mathbf{x}, t) \geq 0, i = 1, ..., \ell\}$ be the feasible set of optimization (29) at time $t, t \in [t_0, t_f]$, i.e., the set constructed by the polynomial constraints of all the risk contours $\hat{\mathcal{C}}_{r_i}^{\Delta}, i = 1, ..., n_{o_s}, \hat{\mathcal{C}}_{r_j}^{\Delta}(t), j = n_{o_s} + 1, ..., n_{o_d}$, as well as the upperbound constraint $M(t)$. Let $\mathbf{x}(t) = \sum_{\alpha=0}^{d} \mathbf{c}_\alpha t^\alpha, t \in [t_1, t_2]$, where $t_0 \leq t_1 \leq t_2 \leq t_f$, be the given trajectory between the two samples $\mathbf{x}_1 \in \mathcal{X}$ and $\mathbf{x}_2 \in \mathcal{X}$ in the uncertain environment. Let $H(t) = \{\mathbf{x} : h(\mathbf{x}, t) \geq 0\}, t \in [t_1, t_2]$, where h is a polynomial and for any $t \in [t_1, t_2]$ the set $H(t)$ is a compact set. Note that $\mathcal{Q}(\mathbf{x}(t), t)$ in the form of (11) is a special case of $H(t)$. Then the following result holds true:

$H(t)$ satisfies the safety constraints of the deterministic optimization in (29) over the time interval $[t_1, t_2]$, i.e.,

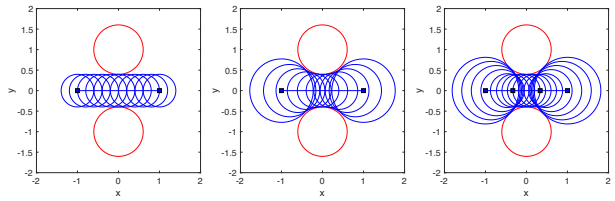


Figure 10. Illustrative Example 5: Left: Constant-sized tube. Middle: Quadratic time-varying-sized tube. Right: Piecewise linear time-varying-sized tube.

$H(t) \subseteq \mathcal{S}(t)$ for all $t \in [t_1, t_2]$, if polynomials $g_i(\mathbf{x}, t), i = 1, ..., \ell$, take the following SOS representation:

$$g_i(\mathbf{x}, t) = \sigma_{0i}(\mathbf{x}, t) + \sigma_{1i}(\mathbf{x}, t)(t - t_1) + \sigma_{2i}(\mathbf{x}, t)(t_2 - t) + \sigma_{3i}(\mathbf{x}, t)h(\mathbf{x}, t) \quad (30)$$

where $\sigma_{0i}(\mathbf{x}, t), \sigma_{1i}(\mathbf{x}, t), \sigma_{2i}(\mathbf{x}, t), \sigma_{3i}(\mathbf{x}, t), i = 1, ..., \ell$, are SOS polynomials with appropriate degrees.

In Algorithm 1, given a fixed tube parameter c, the tube is completed determined, and the corresponding polynomial $h(\mathbf{x}, t)$ is formed. The Verify function verifies the safety of the tube with parameter c via Equation (30). Note that SOS programming for verifying the safety of the tube is a sufficient condition but not a necessary condition. It is possible that a tube is safe but is not verified to be safe via SOS programming. If the tube is verified to be safe, then it must be safe.

Method (2): The second design choice is that we build a trajectory and a tube around the trajectory simultaneously. We use sampling-based motion planning algorithms similar to RRT-SOS in Section 5.1.3 and the one in Section 5.2. When building the RRT tree, any two samples are first connected by a trajectory, and then instead of verifying the safety of the trajectory between two samples, we verify the safety of tubes containing the trajectory between two samples. However, it can be quite expensive to build max-sized tubes between any two samples for the entire RRT tree. So a reasonable design choice is to set a threshold on the minimum required tube size, e.g., the minimum radius r_{min} for balls. Samples that do not admit the minimum tube are rejected. After finding a path from the start point to the goal point, we look for max-sized tube along this path. In summary, Method (2) works as follows: When building the RRT tree, a tube of size r_{min} along the edge between the new node and its nearest tree node is checked. If the tube is verified safe, then the new node with the edge is added to the RRT tree. Otherwise, the new node is discarded. Once we have found a path in the RRT tree from the start point to the goal point, we use Algorithm 1 to search for the max-sized tube along the path. Therefore, the final tube obtained by Method (2) has radius at least as large as r_{min} on every point along the path from the start point to the goal point. This is particularly useful if we do planning directly in the workspace and the robot can be outer approximated by the ball of radius r_{min}.

In Example 7.3, we use Method (1) to directly build the tube on top of a pre-planned trajectory. In Example 7.4, we use Method (2) to build the underlying trajectory and the tube together, satisfying the minimum tube size requirement.

Illustrative Example 5: Consider two round obstacles with radius 0.5, and their centers at $(0, \pm 1)$ on average with Gaussian noise $N(\mu = 0, \sigma^2 = 0.001)$ in both x and y direction. Figure 10 plots the 0.1-risk contours in red. We are going to build a tube between the points $\mathbf{x}(0) = (-1, 0)$ and $\mathbf{x}(1) = (1, 0)$, and the underlying trajectory $\bar{\mathbf{x}}(t) = \mathbf{x}(0) + (\mathbf{x}(1) - \mathbf{x}(0))t = (-1 + 2t, 0)$ for $t \in [0, 1]$. We can consider (1) constant-sized tube, (2) quadratic time-varying-sized tube, and (3) piecewise linaer time-varying-sized tube.

The constant-sized tube is parameterized as $\mathcal{Q}(\bar{\mathbf{x}}(t), t) = \{\mathbf{x} : c^2 - ||\mathbf{x} - \bar{\mathbf{x}}(t)||_2^2 \geq 0\}, t \in [0, 1]$. We search the parameter c using Algorithm 1, and the maximum $c = 0.3980$.

The quadratic time-varying-sized tube is parameterized as $\mathcal{Q}(\bar{\mathbf{x}}(t), t) = \{\mathbf{x} : (a(t - b)^2 + c)^2 - ||\mathbf{x} - \bar{\mathbf{x}}(t)||_2^2 \geq 0, a > 0, 0 \leq b \leq 1, c \geq 0\}$, $t \in [0, 1]$, where the radius $r(t) = a(t - b)^2 + c$ is a quadratic function in t. The parameter $a > 0$ indicates the change rate of the radius. The larger a is, the faster the radius changes over $t \in [0, 1]$. The parameter $b \in [0, 1]$ represents the axis of symmetry of the parabola. $r(t)$ decreases over $t \in [0, b]$ and increases over $t \in [b, 1]$. The parameter c is the value of the minimum point on the parabola. In this example, we fix $a = 1.5, b = 0.5$, and use Algorithm 1 to search for the parameter c. We get the maximum $c = 0.3980$. The quadratic time-varying-sized tube covers more safe regions than the constant sized tube.

Finally we consider piecewise linear time-varying-sized tube. We break the line segment between $(-1, 0)$ and $(1, 0)$ into three smaller line segments, by add two waypoints at $(-1/3, 0)$ and $(1/3, 0)$. On the line segment between $(1/3, 0)$ and $(1, 0)$, we consider a linear time-varying-sized tube of the form $\mathcal{Q}(\bar{\mathbf{x}}(t), t) = \{\mathbf{x} : (at + c)^2 - ||\mathbf{x} - \bar{\mathbf{x}}(t)||_2^2 \geq 0, a > 0, c \geq 0\}$, $t \in [0, 1]$. The radius $r(t) = at + c$ is linear in t. We fix $a = 0.5$ and use Algorithm 1 to search for the parameter c and get the maximum $c = 0.3122$. Similarly, on the line segment between $(-1, 0)$ and $(-1/3, 0)$, we consider a linear time-varying-sized tube of the form $\mathcal{Q}(\bar{\mathbf{x}}(t), t) = \{\mathbf{x} : (a(1 - t) + c)^2 - ||\mathbf{x} - \bar{\mathbf{x}}(t)||_2^2 \geq 0, a > 0, c \geq 0\}, t \in [0, 1]$. We fix $a = 0.5$ and by symmetry get maximum $c = 0.3122$. On the line segment between $(-1/3, 0)$ and $(1/3, 0)$, the linear time-varying-sized tube degenerates to constant sized tube and the radius is $c = 0.3980$. Near the endpoints $(-1, 0)$ and $(1, 0)$ the piecewise linear time-varying-sized tube covers more safety space than the quadratic time-varying-sized tube, while near the waypoints $(-1/3, 0)$ and $(1/3, 0)$, the quadratic time-varying-sized tube covers more safety space.

This example shows that different parametrizations of the tube lead to different max-sized tubes. There is a trade-off between the complexity of the tube parametrization and the safe region the tube covers. The constant-sized tube has the simplest parametrization and it covers the smallest safe region. The quadratic time-varying-sized tube and the piecewise linear time-varying-sized tube have slightly more complicated parametrizations and they cover more safe regions.

7 Experiments

In this section, numerical examples are presented to illustrate the performance of the proposed approaches. To obtain

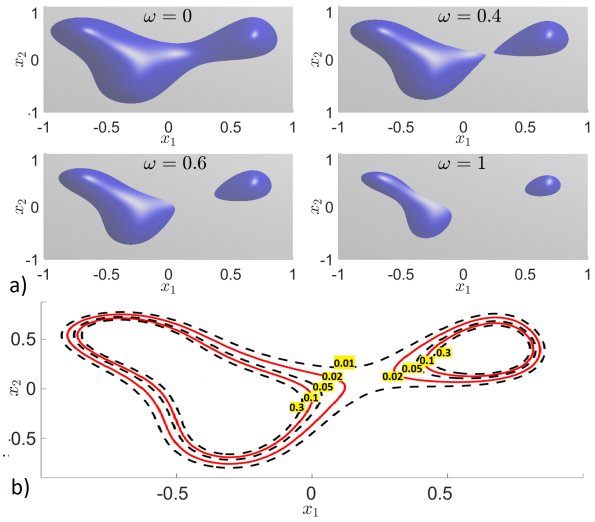


Figure 11. Example 7.1.1: a) Nonconvex obstacle with uncertain parameter $\omega \sim Beta(9, 0.5)$, b) Obtained static risk contours $\hat{\mathcal{C}}_r^{\Delta}$ defined in (16) for different risk levels $\Delta = [0.3, 0.1, 0.05, 0.02, 0.01]$.

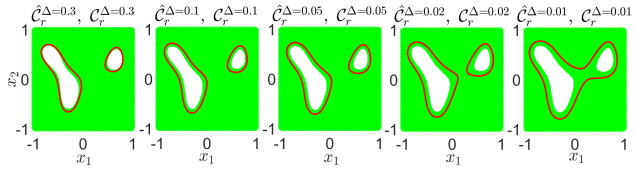


Figure 12. Example 7.1.1: True Δ-risk contour $\mathcal{C}_r^{\Delta}$ (green) and its inner approximation $\hat{\mathcal{C}}_r^{\Delta}$ (outside of the solid-line). Inside the risk contours, probability of collision with the uncertain obstacle is less or equal to the associated risk level Δ.

the risk bounded continuous-time trajectories, we use the time-varying SOS optimization and the RRT-SOS algorithm described in Section 5. Note that the provided RRT-SOS algorithm uses a naive tree search as explained in Section 5. The main objective of the provided RRT-SOS algorithm is to demonstrate how the risk contours in (16) and (22) and the SOS-based continuous-time safety constraints in (25) can be incorporated into sampling-based motion planning algorithms to look for guaranteed risk bounded continuous-time trajectories in stochastic environments. The computations in this section were performed on a computer with Intel i7 2.7 GHz processors and 8 GB RAM. We use the Spotless MATLAB package Tobenkin et al. (2013) to verify the SOS-based continuous-time safety constraints in the RRT-SOS algorithm and the Julia package provided by Khadir et al. (2021) to solve the time-varying SOS optimization.

7.1 Risk Contours

The purpose of this example is to demonstrate how the provided approach can be used to compute the risk contours in the presence of highly complex uncertain unsafe regions.

7.1.1 2D Uncertain Obstacle Static uncertain obstacle of the form (4) is described by the polynomial $\mathcal{P}(\mathbf{x}, \omega) = -0.42x_1^5 - 1.18x_1^4x_2 - 0.47x_1^4 + 0.3x_1^3x_2^2 - 0.57x_1^3x_2 + 0.6x_1^3 - 0.65x_1^2x_2^3 + 0.17x_1^2x_2^2 + 1.87x_1^2x_2 + 0.06x_1^2 + 0.69x_1x_2^4 - 0.14x_1x_2^3 - 0.85x_1x_2^2 + 0.6x_1x_2 - 0.21x_1 + 0.01x_2^5 - 0.06x_2^4 - 0.07x_2^3 - 0.41x_2^2 - 0.08x_2 + 0.07 - 0.1w$ where the uncertain parameter ω has a Beta distribution with parameters $(9, 0.5)$ over $[0, 1]$. Figure 11 shows the uncertain obstacle for different values of the uncertain parameter ω. We obtain the static risk contours defined in (16) for different risk levels Δ as shown in Figure 11 and 12.

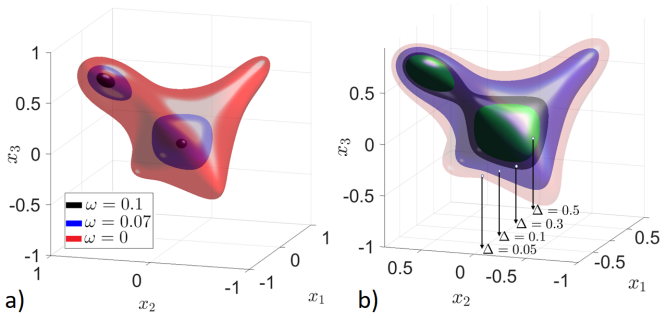


Figure 13. Example 7.1.2: a) Nonconvex obstacle with uncertain parameter $\omega \sim \mathcal{N}(0.1, 0.001)$, b) Obtained static risk contours $\hat{\mathcal{C}}_r^{\Delta}$ defined in (16) for different risk levels $\Delta = [0.5, 0.3, 0.1, 0.05]$.

7.1.2 3D Uncertain Obstacle Static uncertain obstacle of the form (4) is described by the polynomial $\mathcal{P}(\mathbf{x}, \omega) = 0.94 - 0.002x_1 - 0.004x_2 - 0.04x_3 - 0.38x_1^2 + 0.04x_1x_2 - 0.31x_2^2 - 0.05x_1x_3 - 0.01x_2x_3 - 0.4x_3^2 - 0.1x_1^3 - 0.02x_1^2x_2 + 0.09x_1x_2^2 - 0.05x_2^3 + 0.14x_1^2x_3 - 1.83x_1x_2x_3 + 0.11x_2^2x_3 - 0.1x_1x_3^2 + 0.12x_2x_3^2 + 0.34x_3^3 - 0.32x_1^4 - 0.13x_1^3x_2 + 0.48x_1^2x_2^2 + 0.11x_1x_2^3 - 0.34x_2^4 + 0.03x_1^3x_3 + 0.01x_1^2x_2x_3 - 0.005x_1x_2^2x_3 - 0.05x_2^3x_3 + 0.54x_1^2x_3^2 - 0.06x_1x_2x_3^2 + 0.48x_2^2x_3^2 + 0.008x_1x_3^3 + 0.06x_2x_3^3 - 0.3x_3^4 + 0.12x_1^5 + 0.005x_1^4x_2 - 0.1x_1^3x_2^2 + 0.007x_1^2x_2^3 + 0.005x_1x_2^4 + 0.071x_2^5 - 0.02x_1^4x_3 + 0.73x_1^3x_2x_3 - 0.07x_1^2x_2^2x_3 + 0.72x_1x_2^3x_3 - 0.20x_2^4x_3 + 0.03x_1^3x_3^2 - 0.01x_1^2x_2x_3^2 + 0.02x_1x_2^2x_3^2 - 0.05x_2^3x_3^2 - 0.07x_1^2x_3^3 + 0.73x_1x_2x_3^3 + 0.09x_2^2x_3^3 + 0.03x_1x_3^4 - 0.06x_2x_3^4 - 0.31x_3^5 - \omega - 0.84$ where the uncertain parameter ω has a normal distribution with mean 0.1 and variance 0.001, Jasour et al. (2018). Figure 13 shows the uncertain obstacle for different values of the uncertain parameter ω. We obtain the static risk contours defined in (16) for different risk levels Δ as shown in Figure 13.

7.2 Risk Bounded Lane Changing for Autonomous Vehicles

In this example, we generate a risk bounded trajectory for the lane-change maneuver of an autonomous vehicle in the presence of surrounding vehicles. In this scenario, uncertain locations of the surrounding vehicles are modeled as the following sets: $\mathcal{X}_{obs_1}(\omega_1, t) = \{(x_1, x_2) : 0.3^2 - (x_1 - p_1(t, \omega_1))^2 - (x_2 - 1)^2 \geq 0\}$ and $\mathcal{X}_{obs_2}(\omega_2, t) = \{(x_1, x_2) : 0.3^2 - (x_1 - p_2(t, \omega_1))^2 - x_2^2 \geq 0\}$ where $p_1(t, \omega_1) = t + 0.4 + \omega_1$ and $p_2(t, \omega_2) = 2t + 0.6 + \omega_2$ are the uncertain trajectories of the surrounding vehicles with uncertain parameters $\omega_i \sim Uniform[-0.1, 0.1], i = 1, 2$. For the lane-change maneuver, we look for a risk bounded trajectory between the points $\mathbf{x}(0) = (0, 0)$ and $\mathbf{x}(1) = (2, 0)$ over the planning time horizon

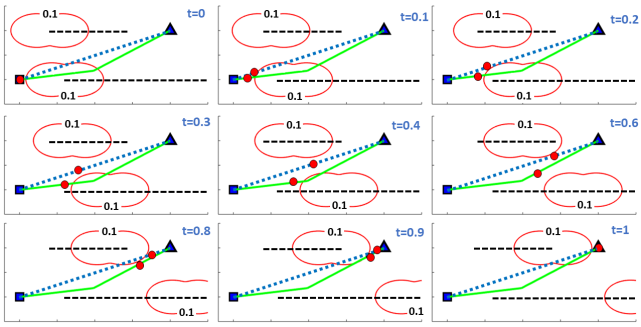


Figure 14. Example 7.2: Risk bounded trajectories for the lane-change maneuver between the start (square) and goal (triangle) points obtained via the time-varying SOS optimization (solid line) and RRT-SOS algorithm (dotted line) in the presence of surrounding vehicles with uncertain trajectories. Dashed lines show the expected values of the uncertain trajectories of the surrounding vehicles.

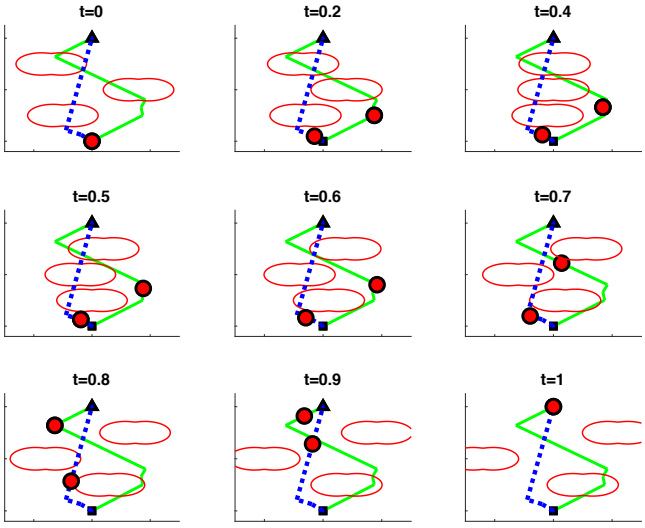


Figure 15. Example 7.3.1: Risk bounded trajectories for a delivery robot between the start (square) and goal (triangle) points obtained via the time-varying SOS optimization (solid line) and RRT-SOS algorithm (dotted line) in the presence of moving uncertain obstacles (red curves).

$t \in [0, 1]$. Figure 14 shows the obtained trajectory using the time-varying SOS optimization and RRT-SOS algorithm considering the dynamic 0.1-risk contours of the surrounding vehicles. The run-time for the time-varying SOS optimization and RRT-SOS are roughly 1.2 and 6.5 seconds, respectively.

7.3 Risk Bounded Trajectory Planning and Tube Design for Delivery Robots

7.3.1 Risk Bounded Trajectory Planning In this example, we generate a risk bounded trajectory for a delivery robot in the presence of uncertain moving obstacles. In this scenario, uncertain locations of the moving obstacles are modeled as the following sets: $\mathcal{X}_{obs_1}(\omega_1, t) = \{(x_1, x_2) : 0.4^2 - (-x_1 + p_1(t, \omega_1))^2 - (x_2 - 1)^2 \geq 0\}$, $\mathcal{X}_{obs_2}(\omega_2, t) = \{(x_1, x_2) : 0.4^2 - (x_1 + p_2(t, \omega_1))^2 - (x_2 - 2)^2 \geq 0\}$, and $\mathcal{X}_{obs_3}(\omega_3, t) = \{(x_1, x_2) : 0.4^2 - (-x_1 + p_3(t, \omega_3))^2 - (x_2 - 3)^2 \geq 0\}$ where $p_1(t, \omega_1) = t - 0.5 + \omega_1$, $p_2(t, \omega_1) = 2t - \omega_2 - 0.8$, and $p_3(t, \omega_3) = 1.8t - 0.7 + \omega_3$ are the uncertain trajectories of the moving obstacles with uncertain parameters $\omega_i \sim Uniform[-0.1, 0.1], i = 1, 2, 3$. We look for a risk bounded trajectory between the start and destination points, $\mathbf{x}(0) = (0, 0)$ and $\mathbf{x}(1) = (0, 4)$, over the planning time horizon $t \in [0, 1]$. Figure 15 shows the obtained trajectories using the time-varying SOS optimization and RRT-SOS algorithm considering the dynamic 0.1-risk contours of the moving obstacles. The run-time for the time-varying SOS optimization and RRT-SOS are roughly 7 and 197 seconds, respectively.

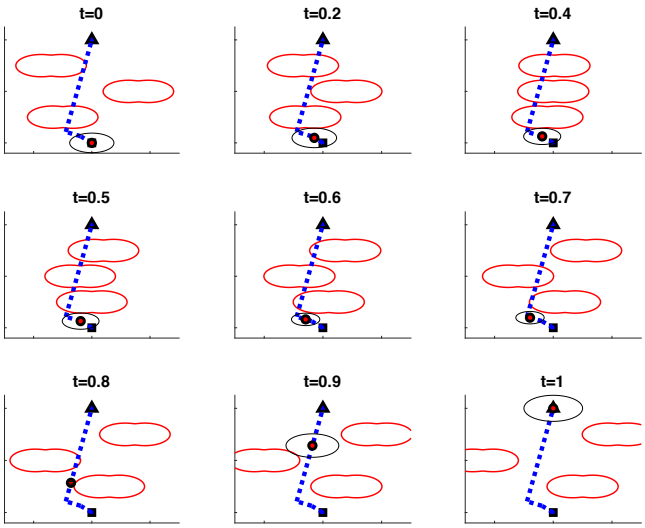


Figure 16. Example 7.3.2: The risk bounded tube (black circles) for a delivery robot (red dot) between the start (square) and goal (triangle) points. The tube is directly built on top of the trajectory obtained via RRT-SOS algorithm (dotted line).

7.3.2 Risk Bounded Tube Design In this example, we build a max-sized tube directly on top of the piecewise-linear trajectory obtained by RRT-SOS in 7.3.1. The entire trajectory consists of four line segments. For each line segment except the last one, we build a constant sized tube around the trajectory. For the last line segment, part of the trajectory is very close to the risk contours. If we build a constant tube on this segment, the tube size would be very small on the entire line segment and hence conservative. So we subdivide the line segment into 3 pieces, and build a constant tube around each piece. Figure 16 shows the tube and the underlying trajectory. This is an example where we build a tube after planning a trajectory, and the tube size can be quite small when the trajectory is very close to risk contours.

7.4 Planning in Cluttered Uncertain Environments

In this example, we look for risk bounded trajectories and risk bounded tubes in cluttered static and dynamic uncertain environments.

7.4.1 2D Static Environment Trajectory Planning In this scenario, we use the RRT-SOS algorithm to obtain a risk bounded trajectory with $\Delta = 0.1$ between the start and goal points, $\mathbf{x}(0) = (0, 0)$, $\mathbf{x}(1) = (5, 5)$, in the presence of circle-shaped obstacles with uncertain position as shown in

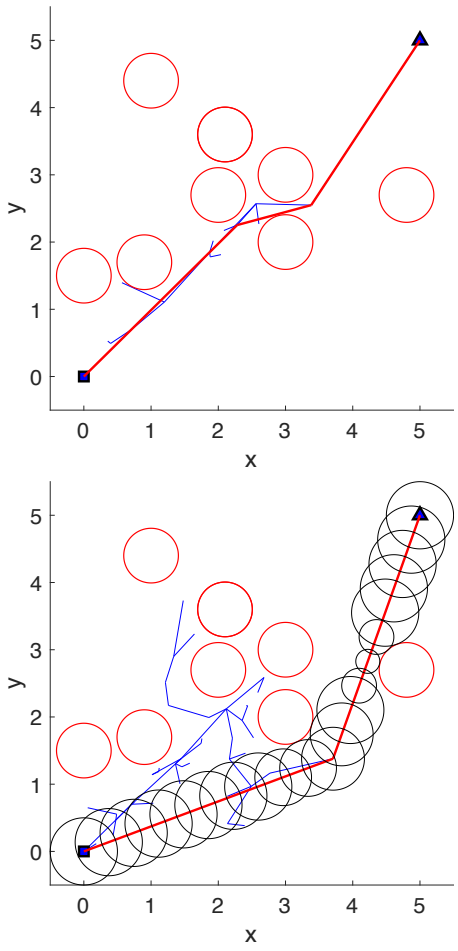


Figure 17. Top: Example 7.4.1. Risk bounded trajectory between the start (square) and goal (triangle) points obtained via the RRT-SOS algorithm in the static uncertain cluttered environment. Red circles are risk contours of obstacles. The blue line segments are the edges of RRT tree. The red line segments are the final path. Bottom: Example 7.4.2. Risk bounded tube (black circles) between the start and goal points. The size of the tube at any point along the trajectory is at least 0.1.

Figure 17. The position of the obstacles is subjected to an additive zero mean Gaussian noise with 0.001 variance. The run-time to obtain the feasible and optimal trajectories are roughly 27 and 92 seconds, respectively.

7.4.2 2D Static Environment Tube Design We look for tubes with minimum size 0.1 and maximum size 0.5 between the start and goal points in the 2D static environment described in 7.4.1. The trajectory planned in 7.4.1 does not meet the requirement, because part of the trajectory is almost tangent to the risk contour. A new underlying trajectory is planned together with the tube. The tube consists of several pieces, each having either constant size or quadratic time-varying size.

7.4.3 Comparison with Chance-Constrained RRT Method We compare our tube design method, for example Method (2), with Chance-Constrained RRT (CC-RRT) method proposed in Luders et al. (2010). In their setting, the system dynamics is linear and has Gaussian noise. The initial position of the system has Gaussian distribution. The obstacles are convex polyhedra. We modify the CC-RRT method to exclude system dynamics. We assume the system has an initial Gaussian distribution, and the distribution does not change over time.

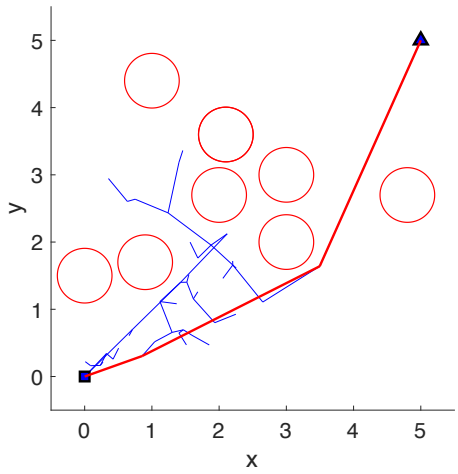


Figure 18. CC-RRT: The planned trajectory is represented by the red line segments. The blue square represents the expected initial position, and the blue triangle represents the expected goal position. The initial position of the system has Gaussian distribution with standard deviation 0.05. The red circles are obstacles, and the blue line segments are the RRT tree. The planned trajectory has bounded risk of 0.1.

We plan a trajectory using CC-RRT in a clustered environment as shown in Figure 18. The planned trajectory is represented by the red line segments. The initial position of the system has Gaussian distribution with standard deviation 0.05. The red circles are obstacles, and the blue line segments are the RRT tree. The planned trajectory has bounded risk of 0.1.

Our method has three advantages over this simplified version of CC-RRT, which does not consider system dynamics: i) Our method can ensure the safety of the trajectory and the tubes over the entire continuous time interval, while CC-RRT can only check certain discrete points on the planning horizon. ii) Our method can deal with more general probabilistic distributions, while CC-RRT only works with Gaussian distribution. iii) Our method can deal with more general obstacles, and does not assume obstacles are convex.

7.4.4 3D Dynamic Environment Trajectory Planning In this scenario, we use the RRT-SOS algorithm to obtain a risk bounded trajectory with $\Delta = 0.1$ between the start and goal points, $\mathbf{x}(0) = (-1, -1, -1)$, $\mathbf{x}(1) = (1, 1, 1)$, in the presence of moving sphere-shaped obstacles with uncertain radius and uncertain trajectories as shown in Figure 19. Radius of the obstacles has a uniform distribution over $[0.1, 0.2]$. Also, trajectories of the obstacles are subjected to an additive zero mean Gaussian noise with 0.001 variance. The run-time to obtain the risk bounded trajectory is roughly 29 seconds.

7.4.5 3D Dynamic Environment Tube Design We look for tubes with minimum size 0.1 between the start and goal points in the 3D dynamic environment described in 7.4.4. The trajectory planned in 7.4.4 does not meet the requirement. A new underlying trajectory is planned together

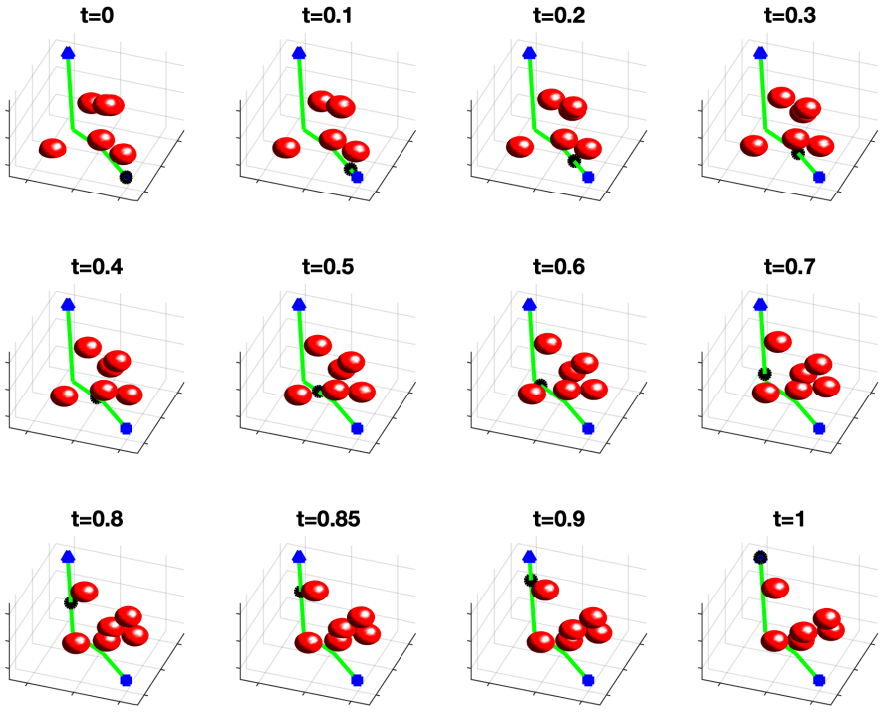


Figure 19. Example 7.4.4: Risk bounded trajectory between the start (square) and goal (triangle) points obtained via the RRT-SOS algorithm in the dynamic uncertain cluttered environment. Red balls are risk contours of obstacles.

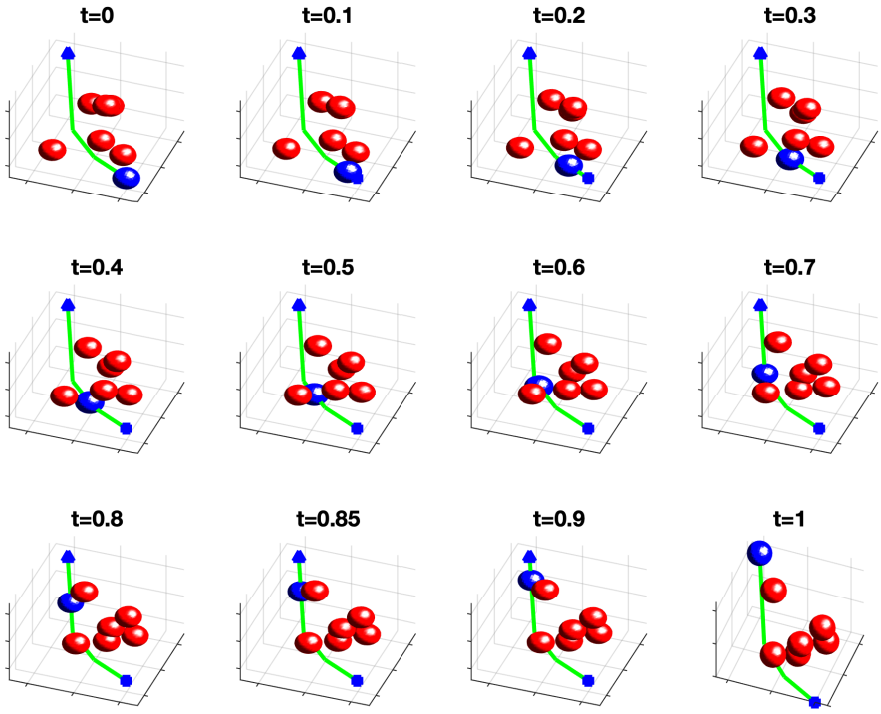


Figure 20. Example 7.4.5: Risk bounded tube (blue balls) whose size is at least 0.1 anywhere between the start (square) and goal (triangle) points in the dynamic uncertain cluttered environment.

with the tube as shown in Figure 20. The tube consists of three parts, each having constant size.

7.5 Discussion

The provided risk bounded algorithms not only are faster than Monte Carlo-based risk bounded RRT algorithms (illustrative example 3), but also provide continuous-time trajectories with guaranteed bounded risk. In addition, compared with the RRT-SOS algorithm, the time-varying SOS optimization is generally faster. The run-time of the time-varying SOS optimization is a function of the order of polynomials of risk contours, the number of the linear pieces of risk bounded trajectories, and the number of iterations of the heuristic algorithm. We use the heuristic algorithm introduced by Khadir et al. (2021) that trades off theoretical guarantees for more efficiency to avoid large scale time-varying SOS optimization. Hence, the heuristic algorithm may fail to obtain a feasible trajectory as observed in Experiment D. To ensure safety, we verify the trajectory returned by the heuristic algorithm using the SOS condition in (25). In the time-varying SOS optimization, one can initially start with a small number of iterations and linear pieces and then increase the parameters if the obtained trajectory is infeasible. On the other hand, the RRT-SOS algorithm is more robust and always returns a feasible trajectory. The provided SOS-based RRT algorithm uses a naive tree search algorithm to look for the risk bounded trajectories. One can improve the performance and run-time by considering efficient sampling-based motion planning algorithms and high-order polynomial trajectories between the samples.

Our algorithms have several limitations.

1. Regarding the analytical method for obtaining the risk contours, the problem of how good the approximation is is equivalent to the problem of how tight Cantelli's inequality (19) is, "tight" in the sense that the gap between the two sides of the inequality is small. In general, the question is how tight the concentration inequalities, such as Markov's inequality, Chebyshev's inequality, and Cantelli's inequality, are. There are probability distributions where the equality holds in concentration inequalities. However, for a general probability distribution, we do not have any quantitative results on the gap between the two sides of the concentration inequality. We only know that empirically most of the time the concentration inequality works well.
2. We assume that the future trajectories and the uncertainty of the obstacles are known. As explained in Section 3, this is because there are plenty of techniques for predicting future trajectories of obstacles with uncertainty, especially in the field of autonomous driving. The prediction of future trajectories is not guaranteed to be accurate. To mitigate the effect of inaccurate predictions, in the real world implementation, such as on an autonomous driving vehicle, the prediction and the planning should be carried out every few milliseconds, much as in the model predictive control (MPC) fashion, and the system is updated with the latest environment information for re-prediction and re-planning.
3. We assume that the moments of the uncertainty can be calculated and are finite. Our methods cannot deal with probability distributions that do not have finite moments, such as Cauchy distribution. For probability distributions that cannot be calculated analytically, we need to use sampling to approximate the moments.
4. Our algorithms do not consider system dynamics. For the planner that considers stochastic nonlinear dynamics, please refer to Han et al. (2022).

8 Conclusion

In this paper, we provided continuous-time trajectory planning algorithms to obtain risk bounded polynomial trajectories in uncertain nonconvex environments that contain static and dynamic obstacles with probabilistic location, size, and geometry. The provided algorithms leverage the notion of risk contours to transform the probabilistic trajectory planning problem into a deterministic planning problem and use convex methods to obtain the continuous-time trajectories with guaranteed bounded risk without the need for time discretization and uncertainty samples. The provided algorithms are suitable for online and large scale planning problems. In addition, we provided continuous-time tube design algorithms that build max-sized tubes around trajectories so that any states deviating from the nominal trajectory but inside the tube are still guaranteed to have bounded risk. In this work, we do not consider system dynamics. For the future work, we will take system dynamics into consideration. We will also consider more uncertainties in the system, such as uncertainty in initial positions, and uncertainty in system dynamics. We will plan risk bounded trajectories for stochastic nonlinear systems.

References

Ahmadi AA and Khadir BE (2021) Time-varying semidefinite programs. *Mathematics of Operations Research, arXiv:1808.03994* .

Axelrod B, Kaelbling LP and Lozano-Pérez T (2018) Provably safe robot navigation with obstacle uncertainty. *The International Journal of Robotics Research* 37(13-14): 1760–1774.

Bellon A, Henrion D, Kungurtsev V and Marecek J (2021) Time-varying semidefinite programming: Geometry of the trajectory of solutions. *arXiv preprint arXiv:2104.05445* .

Blackmore L and Ono M (2009) Convex chance constrained predictive control without sampling. *AIAA Guidance, Navigation, and Control Conference* .

Blackmore L, Ono M, Bektassov A and Williams BC (2010) A probabilistic particle-control approximation of chance-constrained stochastic predictive control. *IEEE Transactions on Robotics* 26(3): 502–517.

Blackmore L, Ono M and Williams BC (2011) Chance-constrained optimal path planning with obstacles. *IEEE Transactions on Robotics* 27(6): 1080–1094.

Calafiore GC and Campi MC (2006) The scenario approach to robust control design. *IEEE Transactions on Automatic Control* 51(5): 742–753.

Cannon M (2017) Chance-constrained optimization with tight confidence bounds. *arXiv preprint arXiv:1711.03747* .

Chai Y, Sapp B, Bansal M and Anguelov D (2020) Multipath: Multiple Probabilistic Anchor Trajectory Hypotheses for Behavior Prediction. In: *Conference on Robot Learning*. PMLR, pp. 86–99.

Charles R, Bry A and Roy N (2016.) Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments. *Robotics Research, Springer* : vol. 114, 649–666.

Dai S, Schaffert S, Jasour A, Hofmann A and Williams B (2019) Chance constrained motion planning for high-dimensional robots. *IEEE International Conference on Robotics and Automation (ICRA)* .

Dawson C, Jasour A, Hofmann A and Williams B (2020) Provably safe trajectory optimization in the presence of uncertain convex obstacles. *IEEE International Conference on Intelligent Robots and Systems (IROS)* .

Deits R and Tedrake R (2015) Efficient mixed-integer planning for uavs in cluttered environments. *IEEE International Conference on Robotics and Automation (ICRA)* .

Gu J, Sun C and Zhao H (2021) DenseTNT: End-to-end Trajectory Prediction from Dense Goal Sets. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. pp. 15303–15312.

Han W, Jasour A and Williams B (2022) Non-gaussian risk bounded trajectory optimization for stochastic nonlinear systems in uncertain environments. In: *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, pp. 11044–11050.

Henrion D, Lasserre JB and Löfberg J (2009) Gloptipoly 3: moments, optimization and semidefinite programming. *Optimization Methods & Software* 24(4-5): 761–779.

Jacod J and Protter P (2012) *Probability essentials*. Springer Science & Business Media.

Janson L, Schmerling E and Pavone M (2018) Monte carlo motion planning for robot trajectory optimization under uncertainty. *Robotics Research* : 343–361.

Jasour A (2019) Risk aware and robust nonlinear planning (rarnop). *Course Notes for MIT 16.S498* URL rarnop.mit.edu.

Jasour A, Han W and Williams B (2021a) Convex risk bounded continuous-time trajectory planning in uncertain nonconvex environments. *Robotics: Science and Systems* .

Jasour A, Han W and Williams B (2021b) Real-time risk-bounded tube-based trajectory safety verification. In: *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, pp. 4307–4313.

Jasour A, Hofmann A and Williams BC (2018) Moment-sum-of-squares approach for fast risk estimation in uncertain environments. *IEEE Conference on Decision and Control (CDC)* : 2445–2451.

Jasour A and Williams B (2019) Risk contours map for risk bounded motion planning under perception uncertainties. *Robotics: Science and Systems* .

Khadir BE, Lasserre JB and Sindhwani V (2021) Piecewise-linear motion planning amidst static, moving, or morphing obstacles. *IEEE International Conference on Robotics and Automation (ICRA)* .

Kumar V, Zefran M and Ostrowski J (1999) Motion planning and control of robots. *New York: Wiley* .

Kummer EE (1837) De integralibus quibusdam definitis et seriebus infinitis. .

Lasserre JB (2001) Global optimization with polynomials and the problem of moments. *SIAM Journal on optimization* 11(3): 796–817.

Laurent M (2009) Sums of squares, moment matrices and optimization over polynomials. *Emerging Applications of Algebraic Geometry, Springer* : 157–270.

LaValle SM (2006) *Planning algorithms*. Cambridge university press.

Lofberg J (2004) Yalmip: A toolbox for modeling and optimization in matlab. *IEEE International Conference on Robotics and Automation (ICRA)* .

Lu Q, Han W, Ling J, Wang M, Chen H, Varadarajan B and Covington P (2022) Kemp: Keyframe-based hierarchical end-to-end deep model for long-term trajectory prediction. In: *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, pp. 646–652.

Luders B, Kothari M and How J (2010) Chance constrained rrt for probabilistic robustness to environmental uncertainty. *AIAA Guidance, Navigation, and Control Conference* .

Lynch KM and Park FC (2017) *Modern Robotics*. Cambridge University Press.

Nemirovski A and Shapiro A (2007) Convex approximations of chance constrained programs. *SIAM Journal on Optimization* 17(4): 969–996.

Parrilo PA (2003) Semidefinite programming relaxations for semialgebraic problems. *Mathematical programming* 96(2).

Putinar M (1993) Positive polynomials on compact semi-algebraic sets. *Indiana University Mathematics Journal* 42(3): 969–984.

Reif J and Sharir M (1994) Motion planning in the presence of moving obstacles. *Journal of the ACM* 41(4): 764–790.

Roux P, Voronin YL and Sankaranarayanan S (2018) Validating numerical semidefinite programming solvers for polynomial invariants. *Formal Methods in System Design* 53(2): 286–312.

Schwarting W, Alonso-Mora J, Pauli L, Karaman S and Rus D (2017) Parallel autonomy in automated vehicles: Safe motion generation with minimal intervention. *IEEE International Conference on Robotics and Automation (ICRA)* : 1928–1935.

Summers T (2018) Distributionally robust sampling-based motion planning under uncertainty. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* : 6518–6523.

Tobenkin MM, Permenter F and Megretski A (2013) spotless: Polynomial and conic optimization URL github.com/spot-toolbox/spotless.

Varadarajan B, Hefny A, Srivastava A, Refaat KS, Nayakanti N, Cornman A, Chen K, Douillard B, Lam CP, Anguelov D et al. (2022) Multipath++: Efficient information fusion and trajectory aggregation for behavior prediction. In: *2022 International Conference on Robotics and Automation (ICRA)*. IEEE, pp. 7814–7821.

Wang A, Jasour A and Williams BC (2020) Non-gaussian chance-constrained trajectory planning for autonomous vehicles under agent uncertainty. *IEEE Robotics and Automation Letters* 5(4): 6041–6048.